

LKK-HSAT

Recovering Cardinality Structure from CNF
and Deciding It by Max-Flow

*A structural front-end for CDCL SAT solving,
with certified refutations and a correctness-gated methodology*

Ümit Altınöz

Reference campaign	20260817Tphase4cfinal04Z
Measured instances	14 generated, 1 192 public prepared
Correctness cases	46 with 0 failures
Compiled	19 August 2026

Abstract

Conflict-driven clause learning is the dominant technology in propositional satisfiability, but its underlying proof system is resolution, and there are natural constraint families for which resolution provably requires proofs of exponential size. The pigeonhole principle is the canonical example. Such families arise whenever a cardinality constraint is compiled into clauses, which is routine in encodings of allocation, scheduling and configuration problems.

This report presents LKK-HSAT, which closes a loop that the literature has left open. Detecting cardinality constraints hidden in a CNF is a solved problem, but the standard next step is to hand the recovered constraints to a pseudo-Boolean solver or to re-encode them for a CDCL solver; the closest prior work states explicitly that using the recovered information to improve solving time remained future work. LKK-HSAT instead *decides* the recovered structure directly: it assembles exact- r and capacity families into an allocation model, refutes that model by a max-flow argument when Hall’s condition fails, and answers without invoking a solver at all. Because the refutation is combinatorial rather than clausal, it admits a compact certificate, and the report specifies one — a clause database with resolution parents, the accepted families, and a minimum cut — together with a checker that validates it against the original formula. Instances on which every locked baseline exhausts the benchmark budget are decided, with a certificate, in tens of milliseconds.

The evaluation is deliberately adversarial to its own thesis. Two benchmark families exist to expose the cost of the technique where it cannot help, the comparison includes a preprocessor that attacks the same instances by an entirely different route, and a lemma-transfer extension is reported as the negative result it turned out to be. The measured advantage over plain CDCL is not unique to this approach; the certificate about the *original* formula appears to be.

A campaign over 1 192 public instances from SATLIB and the 2024 SAT Competition then fixed the scope by measurement. Correctness held everywhere: across 4770 runs no two configurations returned different definite answers, and none disagreed with the 338 instances whose answer is externally known. Performance did not: plain Kissat is the best arm, every configuration carrying the front end trails it by two to six percent, and switching the structural pipeline from fully disabled to fully enabled moves PAR-2 by 0.03 % without changing the solved count. A control arm shows that what first looked like a structural speedup over CaDiCaL is a property of the parser, not of the recovered structure. The advantage is therefore confined to instances whose obstruction is a counting argument — on the pigeonhole family the front end answers in tens of milliseconds where both baselines exhaust a 600 s budget — and the contribution is a scope established by measurement, not a general speedup.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	The approach in outline	1
1.3	Contributions	2
1.4	Scope of the claims	3
1.5	Structure of the report	3
2	Preliminaries	4
2.1	Formulas, clauses and resolution	4
2.2	Cardinality constraints and their encodings	5
2.3	The pigeonhole principle and proof complexity	5
2.4	Flows, matchings and Hall’s condition	6
2.5	Hidden structure	6
3	Related work	8
3.1	Recovering structure from CNF	8
3.2	Matching and flow reasoning	8
3.3	Symmetry breaking	9
3.4	Pseudo-Boolean solving and cutting planes	9
3.5	Structure detection inside modern solvers	9
3.6	Machine learning and LLM-evolved solvers	9
3.7	Summary of positioning	10
4	The LKK-HSAT pipeline	12
4.1	Design constraints	12
4.2	Stage 1–2: the gates	12
4.3	Stage 3: bounded resolution recovery	13
4.4	Stage 4: the registry	13
4.5	Stage 5: the flow test	13
4.6	Stage 6: witness or fallback	14
5	Certification	15
5.1	What a certificate must establish	15
5.2	The witness format	15
5.3	The checking procedure	16
5.4	What a passing check does not establish	16
5.5	Relation to proof logging in the literature	17
6	The cost of the registry search	18
6.1	The candidate search	18
6.2	Two accounting defects	18
6.2.1	Overcharging: the family test	19
6.2.2	Undercharging: the clique growth	19
6.3	Refitting the budget	19
6.4	Effect	20

7	Transferring recovered structure: a negative result	21
7.1	What may be transferred	21
7.2	Independent verification	21
7.3	Results of the verification campaign	22
7.4	The negative result	22
7.5	What the experiment did reveal	23
8	Methodology	24
8.1	Phase discipline	24
8.2	The correctness gate	24
8.2.1	An incident the gate caught	25
8.3	Benchmark families	25
8.4	Measurement protocol	26
8.5	The prepared public corpus	26
8.6	Accounting invariant tests	27
9	Results	28
9.1	Headline comparison	28
9.2	Where the time goes	29
9.3	Correctness campaign	30
9.4	Reading the numbers correctly	30
10	Comparison with competing techniques	31
10.1	Static symmetry breaking	31
10.2	A qualitative difference: what the proof is about	32
10.3	Techniques not measured	32
10.4	Machine learning approaches	32
10.5	Summary	33
11	Phase 6: Demanding Evidence the Proof Does Not Use	34
11.1	A negative result that pointed at its own cause	34
11.2	The defect	34
11.3	One-sided demands are sound	34
11.4	Effect on the public corpus	35
11.5	What the episode says about the method	36
12	Phase 5: The Public-Benchmark Campaign	37
12.1	What was measured	37
12.2	Correctness	37
12.3	The headline result	37
12.4	What the ablation shows	38
12.5	Separating the parser from the structural analysis	39
12.6	Where the front end does fire	39
13	Threats to validity and limitations	40
13.1	The benchmarks are generated	40
13.2	The baseline may be conservative	40
13.3	A regression stands	40
13.4	The decision procedure is one-sided and incomplete	40
13.5	Budgets bound what can be found	41

13.6 Cost profile	41
13.7 Statistical strength	41
13.8 Scope of the claims	41
14 Conclusion and outlook	42
14.1 What was established	42
14.2 The public corpus, and what it took to fire there	42
14.3 What the public campaign settled	42
14.4 The defensible claim	43
14.5 Immediate obligations	43
14.6 Technical directions	44
14.7 A note on durability	44
A Full correctness data	45
B Complete measurement data	47
C Reproduction	51
C.1 Environment	51
C.2 Commands	51
C.3 Artifacts of a campaign	51
C.4 How this document is built	52
C.5 Data availability	52
D A worked example	53
D.1 The formula	53
D.2 Stage 1: the cheap gate	53
D.3 Stage 3: recovery	53
D.4 Stage 4: the registry	53
D.5 Stage 5: the flow test	54
D.6 The certificate	54
D.7 Contrast: where the procedure declines	54

List of Tables

3.1	Components against established work	10
6.1	Budget selection by F-race	20
7.1	Lemma generation and verification	22
8.1	Development phases	24
8.2	Prepared public corpus	26
9.1	Headline comparison against the sealed engine	28
9.2	Per-stage cost of the front-end	29
10.1	Comparison with symmetry breaking	31
11.1	Pigeonhole: counting versus resolution	35
12.1	Correctness of the campaign	37
12.2	PAR-2 over the evaluation split	38
12.3	PAR-2 over the SATLIB subset	38
12.4	Component ablation	38
12.5	The parser control	39
A.1	Correctness campaign	45
B.1	Complete measurement data	47
C.1	Reproduction commands	51

CHAPTER 1

Introduction

1.1 Motivation

Propositional satisfiability solving has been transformed over three decades by conflict-driven clause learning (CDCL). Modern implementations dispatch industrial formulas with millions of clauses, and the annual SAT Competition has made progress measurable and reproducible. Yet the technique has a boundary that engineering cannot move. CDCL searches for a refutation in the resolution proof system, and by Theorem 2.6 there are natural formulas whose shortest resolution refutation is exponentially large. No implementation, however carefully tuned, can produce a proof object that does not exist.

The formulas exhibiting this behaviour are not pathological curiosities. They arise whenever a cardinality constraint — “exactly one value per variable”, “at most two tasks per machine”, “every job is assigned” — is compiled into clauses. Allocation, scheduling, configuration and graph colouring encodings are full of them. The compilation is lossy in a specific sense: the constraint is still logically present, but the *fact that it is a cardinality constraint* is gone, and with it the possibility of reasoning about it by counting rather than by case analysis.

This report describes a system, LKK-HSAT, built around a single idea: recover that lost structure from the clauses, decide it with an algorithm suited to counting, and fall back to a conventional solver whenever recovery fails.

Recovering the structure is not new, and Chapter 3 traces that lineage carefully. What is new is the second half. The established route, once constraints have been detected, is to hand them to a solver equipped to reason about them — a pseudo-Boolean engine — or to re-encode them into better clauses and hand those back to CDCL. Both keep a search-based solver in the loop. LKK-HSAT takes the recovered constraints and *decides* them: the allocation model they describe is refuted, when it can be, by a maximum-flow computation, and the formula is answered without any solver being invoked.

Two consequences follow, and they are the substance of this report. The decision is combinatorial, so it is fast where the clausal proof is provably not; and it is combinatorial, so it can be certified compactly, by exhibiting a cut rather than a proof. Chapter 5 specifies that certificate and the checker that validates it against the original formula.

1.2 The approach in outline

Given a CNF formula, the system proceeds through six stages, developed formally in Chapter 4:

1. a linear-time gate that looks for a cheap structural signal and declines immediately when there is none;
2. a second gate that refuses expensive work on formulas whose size makes it unprofitable;
3. bounded resolution, which derives consequences under strict depth, length and time budgets, exposing constraints hidden behind auxiliary variables;

4. a registry that assembles exact- r and capacity families, accepting one only when *every* clause the family requires is present;
5. a max-flow test on the resulting allocation model, whose infeasibility refutes the formula;
6. a witness for the independent checker, or a fallback handing the unchanged formula to a locked CDCL solver.

Soundness rests on two observations. Everything derived in stage three is a logical consequence by Lemma 2.2, and the acceptance test in stage four is exact: a family is recognised only when all $\binom{|G|}{r+1} + \binom{|G|}{|G|-r+1}$ of its clauses are present. Unrelated clauses can never count as evidence *against* a candidate, so adding noise to a formula cannot cause a false refutation — it can only make discovery more expensive.

1.3 Contributions

Deciding recovered structure rather than re-encoding it.

Detection of cardinality constraints in CNF was solved by Biere et al. [7], whose paper states that “using that information to improve the run time of the solvers is future work”. The established uses of a detected constraint are re-encoding it or passing it to a pseudo-Boolean solver. This work does neither: it assembles the recovered constraints into an allocation model and refutes that model directly by max-flow (Theorem 4.6), answering the formula with no solver in the loop. That composition — recovery, then combinatorial decision — is what the report contributes.

A certificate for the combinatorial refutation.

Because the refutation is a Hall violation rather than a clausal derivation, it can be certified by exhibiting a cut. Chapter 5 specifies the format — clause database with resolution parents, accepted families, minimum cut — and a checker that re-derives everything from the original CNF. Proposition 5.1 states what a passing check establishes. Crucially the certificate concerns the *original* formula, unlike preprocessing techniques whose proofs concern a transformed instance.

Budget accounting as a correctness property.

Chapter 6 shows that the natural implementation of the registry mischarges its own budget in two opposite directions, and that this, not algorithmic cost, was what prevented an instance from being decided at all. The analysis generalises: a search governed by a work budget is only bounded if the charge tracks the work, and the report gives tests that enforce this by instrumenting the data structure and comparing lookups against the charge.

A measurement regime that catches its own errors.

Every performance number is gated on a correctness campaign that runs first and aborts the campaign on any disagreement. During this work the gate stopped a campaign because the C++ engine and the Python reference had been run under different budgets — a harness defect that would otherwise have entered this report as a false claim of cross-implementation agreement (Section 8.2.1).

A documented negative result.

Chapter 7 reports a lemma-transfer extension in which all 3344 generated candidates

were independently verified and none improved end-to-end runtime, together with the reasons, which differ by family. The component was removed rather than retained.

1.4 Scope of the claims

Three boundaries are stated at the outset because they bound everything that follows.

First, the components have antecedents even though their composition does not. Cardinality recovery from CNF is Biere et al. [7] and, earlier, Ostrowski et al. [6]; matching-based reasoning over an allocation graph is Régim [5]. Chapter 3 gives the mapping in detail. The claim made here is about assembling them into a decision procedure with a certificate, not about inventing any of them.

Second, the measured advantage over plain CDCL is not unique to this system. Chapter 10 shows static symmetry breaking turning the same timeout into tens of milliseconds by an entirely different mechanism. A speed figure quoted against CaDiCaL alone would be true and misleading. Where the two approaches genuinely separate — hidden cardinality, and what the proof is about — is stated there precisely.

Third, the evaluation is on generated benchmarks. A corpus of 1 192 public instances has been acquired, hash-verified and prepared, but the campaign over it has not been run. Nothing here is evidence about real workloads.

1.5 Structure of the report

Chapter 2 fixes notation and states the proof-complexity and flow results the argument uses. Chapter 3 surveys the prior art and competing techniques, and locates this work within them.

The technical core follows. Chapter 4 develops the pipeline formally and proves the refutation theorem; Chapter 5 specifies the certificate and its checker; Chapter 6 analyses the cost of the registry search and the two budget defects found in it; and Chapter 7 reports the lemma-transfer experiment and its negative outcome.

The evaluation then follows. Chapter 8 describes the correctness regime and the benchmark families, Chapter 9 presents the measurements, and Chapter 10 measures the system against symmetry breaking and discusses machine-learning approaches. Chapter 13 states the threats to validity and Chapter 14 the outlook.

Appendix A carries the full correctness and measurement data, Appendix C the reproduction commands, and Appendix D a complete worked example tracing every definition of Chapter 4 on a small instance.

CHAPTER 2

Preliminaries

This chapter fixes notation and states the results the rest of the report relies on. Readers familiar with propositional proof complexity and network flows may wish to skim to Section 2.2, which introduces the encoding families that the front-end is designed to recover.

2.1 Formulas, clauses and resolution

Let V be a finite set of Boolean variables. A *literal* is a variable $v \in V$ or its negation $\neg v$; we write $\text{var}(\ell)$ for the variable underlying a literal ℓ . A *clause* is a finite set of literals, read disjunctively, and a formula in conjunctive normal form (CNF) is a finite set of clauses, read conjunctively. We write F for a CNF formula and $\mathcal{C}(F)$ for its clause set.

A clause is *tautological* if it contains both v and $\neg v$ for some variable v . Throughout, clauses are kept in a canonical form: literals are sorted by variable index with the positive literal first, duplicates are removed, and tautological clauses are discarded. This makes clause identity a syntactic test, which matters in Chapter 4 where membership of a clause in a recovered database is the acceptance criterion for structure.

A clause is *monotone positive* if all its literals are positive and it contains at least two of them, and *monotone negative* if all its literals are negative under the same size condition. These two classes carry the cardinality signal the front-end looks for.

Definition 2.1 (Resolution). Let C_1 and C_2 be clauses with $\ell \in C_1$ and $\neg\ell \in C_2$, and suppose ℓ is the only variable on which they clash. The *resolvent* of C_1 and C_2 on the pivot ℓ is

$$\text{Res}_\ell(C_1, C_2) = (C_1 \setminus \{\ell\}) \cup (C_2 \setminus \{\neg\ell\}).$$

Resolution is sound: any assignment satisfying C_1 and C_2 satisfies their resolvent. It is also refutationally complete for CNF, so a formula is unsatisfiable exactly when the empty clause is derivable. This is the proof system that conflict-driven clause learning searches in, a point Section 2.3 returns to.

Lemma 2.2 (Soundness of derived clauses). *Let $\mathcal{D}$ be a set of clauses obtained from $\mathcal{C}(F)$ by repeated resolution. Then $F \models D$ for every $D \in \mathcal{D}$.*

Proof. By induction on the derivation depth. A clause at depth 0 belongs to $\mathcal{C}(F)$ and is entailed trivially. For the inductive step, let D be the resolvent of C_1 and C_2 on pivot ℓ , with $F \models C_1$ and $F \models C_2$ by hypothesis. Take any assignment α satisfying F . Then α satisfies both C_1 and C_2 . If $\alpha(\ell)$ is true then α must satisfy C_2 through some literal other than $\neg\ell$, and that literal lies in D ; if $\alpha(\ell)$ is false then α satisfies C_1 through a literal other than ℓ , which also lies in D . Either way α satisfies D . $\square$

Lemma 2.2 is the reason the recovery stage of Chapter 4 can be heuristic and budgeted without endangering soundness: whatever it derives is entailed, so structure built out of derived clauses is structure the original formula already implies.

2.2 Cardinality constraints and their encodings

Definition 2.3 (Cardinality constraint). For literals $\ell_1, \dots, \ell_n$, an integer threshold k and a relation $\otimes \in \{<, \leq, =, \geq, >\}$, the cardinality constraint $\sum_{i=1}^n \ell_i \otimes k$ is satisfied by an assignment exactly when the number of literals it makes true stands in relation $\otimes$ to k .

Cardinality constraints do not belong to the language of CNF, so a solver that consumes CNF only ever sees an encoding of them. The simplest encoding of $\sum_{v \in G} v \leq k$ over a set of variables G forbids every way of exceeding the threshold:

$$\text{AtMost}_k(G) = \left\{ \bigvee_{v \in S} \neg v : S \subseteq G, |S| = k + 1 \right\}, \quad |\text{AtMost}_k(G)| = \binom{|G|}{k+1}. \quad (2.1)$$

Dually, $\sum_{v \in G} v \geq r$ is expressed by forbidding every way of falling short:

$$\text{AtLeast}_r(G) = \left\{ \bigvee_{v \in S} v : S \subseteq G, |S| = |G| - r + 1 \right\}, \quad |\text{AtLeast}_r(G)| = \binom{|G|}{|G| - r + 1}. \quad (2.2)$$

This is the *binomial* or *pairwise* encoding. Its size is polynomial for fixed k but grows quickly with the threshold, which is why practical encodings introduce auxiliary variables; Section 2.5 returns to that case.

Combining the two directions gives the object the front-end recovers.

Definition 2.4 (Exact- r family). For a variable set G with $0 < r < |G|$, the *exact- r family* over G is

$$\mathcal{F}_{=r}(G) = \text{AtMost}_r(G) \cup \text{AtLeast}_r(G),$$

whose models are exactly the assignments making precisely r variables of G true. Its size is

$$|\mathcal{F}_{=r}(G)| = \binom{|G|}{r+1} + \binom{|G|}{|G| - r + 1}. \quad (2.3)$$

Example 2.5. For $|G| = 11$ and $r = 2$, Equation (2.3) gives $\binom{11}{3} + \binom{11}{10} = 165 + 11 = 176$ clauses. This number recurs in Chapter 6: it is the price of testing a single exact- r candidate at that size, and mischarging it was the defect that kept the engine from deciding one of the benchmark instances at all.

2.3 The pigeonhole principle and proof complexity

The pigeonhole principle PHP_n^m asserts that m pigeons cannot be placed injectively into $n < m$ holes. With a variable $x_{i,j}$ for “pigeon i occupies hole j ”, it is the conjunction of

$$\underbrace{\bigwedge_{i=1}^m \bigvee_{j=1}^n x_{i,j}}_{\text{every pigeon is placed}} \quad \wedge \quad \underbrace{\bigwedge_{j=1}^n \bigwedge_{1 \leq i < i' \leq m} (\neg x_{i,j} \vee \neg x_{i',j})}_{\text{no hole is shared}}. \quad (2.4)$$

The left conjunct is AtLeast_1 over each pigeon’s row; the right is AtMost_1 over each hole’s column. The formula is therefore a pure instance of the encodings of Section 2.2, and this is exactly the shape the front-end searches for.

Two classical results explain both why CDCL struggles here and why the technique in this report can help at all.

Theorem 2.6 (Haken [1]). *Every resolution refutation of PHP_n^{n+1} has size $2^{\Omega(n)}$.*

Theorem 2.7 (Cook, Coullard and Turán [2]). *PHP_n^{n+1} admits cutting-plane refutations of size polynomial in n .*

Because CDCL searches for a resolution refutation, Theorem 2.6 is a statement about CDCL itself rather than about any particular implementation: no amount of engineering removes an exponential lower bound on the object being searched for. Theorem 2.7 shows the obstacle is an artefact of the proof system, not of the problem. Hooker [3] connects the two by showing that generalised resolution over cardinality constraints is a restricted form of cutting planes, which is why a pseudo-Boolean solver given the constraints — rather than their clausal encoding — refutes pigeonhole in polynomial time.

Remark 2.8. This framing determines what the present work can honestly claim. Recovering the cardinality structure and deciding it combinatorially *reaches* the capability that Theorem 2.7 guarantees; it does not exceed it. Chapter 3 develops this comparison, and Chapter 13 states the consequences for the claims.

2.4 Flows, matchings and Hall’s condition

The decision procedure of Section 4.5 is a max-flow computation, so we recall the standard apparatus.

A *flow network* $N = (W, A, \text{cap}, s, t)$ consists of a directed graph (W, A) , a non-negative integer capacity on each arc, a source s and a sink t . A *flow* assigns to each arc a non-negative value not exceeding its capacity such that, at every node other than s and t , inflow equals outflow. Its *value* is the net flow leaving s . An s - t *cut* partitions W into $S \ni s$ and $T \ni t$; its capacity is the total capacity of arcs from S to T .

Theorem 2.9 (Max-flow min-cut). *The maximum value of an s - t flow equals the minimum capacity of an s - t cut. When all capacities are integral, a maximum flow with integral values exists.*

Integrality matters here: the allocation model of Section 4.4 is meaningful only if flow values can be read as discrete assignments of variables to resources.

Theorem 2.10 (Hall [4]). *A bipartite graph with parts X and Y admits a matching saturating X if and only if $|N(S)| \geq |S|$ for every $S \subseteq X$, where $N(S)$ is the neighbourhood of S in Y .*

Hall’s condition is the combinatorial content of the refutation in Theorem 4.6: a violated instance exhibits a set of demands that collectively exceed the capacity reachable from them, and the minimum cut returned by Theorem 2.9 localises exactly such a set. Régis [5] made this the basis of the standard filtering algorithm for the `alldifferent` constraint in constraint programming, a lineage Chapter 3 discusses.

2.5 Hidden structure

The encodings of Section 2.2 are rarely emitted verbatim by a modelling tool. Practical encoders introduce auxiliary variables to reduce clause count, so the cardinality constraint present in the designer’s model is not present in the CNF as any set of clauses over the original variables. We call such a constraint *hidden*.

A representative gadget replaces a clause C by two clauses over a fresh variable $y \notin \text{var}(C)$:

$$C \rightsquigarrow (C' \vee y) \wedge (C'' \vee \neg y), \quad C' \cup C'' = C. \quad (2.5)$$

Resolving the two on y recovers $C' \vee C''$, so the original clause is a consequence of the pair but appears nowhere in the formula. A syntactic search over the CNF finds nothing; a search over the resolution closure finds the constraint. This observation motivates the recovery stage of Section 4.3, and it is also the mechanism used to generate the *hidden pigeonhole* benchmark family of Chapter 8.

We stress that recovering hidden cardinality constraints is itself established work: Biere et al. [7] detect them behind the two-product and nested encodings, and their semantic method based on unit propagation extracts AtMost_k for arbitrary k . Chapter 3 positions the present work against that baseline.

CHAPTER 3

Related work

The families on which this system performs well are precisely the families the literature has targeted for two decades. This chapter maps each component onto its established counterpart. It is placed before the results deliberately: the measurements of Chapter 9 should be read against this background, not after it.

3.1 Recovering structure from CNF

The idea that a CNF retains recoverable structure lost during encoding goes back at least to Ostrowski et al. [6], who extract equivalences and gate definitions and exploit them during search.

The direct predecessor of the present recovery stage is Biere, Le Berre, Lonca and Manthey [7]. They give two detection methods. The *syntactic* method analyses the binary-clause implication graph to extract AtMost_1 and AtMost_2 constraints very cheaply. The *semantic* method uses unit propagation and detects AtMost_k for *arbitrary* k , at higher cost. The second is strictly more general than the exact- r detection of Section 4.4, which is confined to families expressible through the binomial encoding of Definition 2.4.

Crucially, their work already addresses hidden constraints. They present an algorithm recovering AtMost_1 constraints encoded through the two-product construction, remarking that no existing system detected constraints encoded that way. The hidden pigeonhole family used in this report is the same phenomenon under a different gadget. Any suggestion that recovering hidden cardinality is novel here is therefore incorrect, and Chapter 10 confines the claim accordingly.

Their reported effect is substantial: the smallest unsolved unsatisfiable instances of the SAT 2009 competition (**sgen**) are solved by Sat4j within seconds once the AtMost_2 constraints are revealed, and a benchmark that no solver had solved in a day falls in one second after 22 AtMost_2 and 20 AtMost_3 constraints are exposed.

What they leave open is the half this report supplies. Their conclusion states that revealing constraints on large application benchmarks was achieved, but that “using that information to improve the run time of the solvers is future work”. The present system is an attempt at exactly that: an integrated engine whose end-to-end cost, including the cost of finding nothing, is measured.

The line continues. Reeves [13] devotes a doctoral thesis to cardinality constraints in SAT solving, covering reencoding, native cardinality reasoning inside CDCL, and — directly relevant to the witness argument of Section 4.5 — proof logging for cardinality reasoning.

3.2 Matching and flow reasoning

The decision procedure of Theorem 4.6 is a max-flow computation over an allocation graph. In constraint programming this is standard: Régim [5] bases the filtering algorithm for **alldifferent** on bipartite matching in a value graph, using Hall’s theorem (Theorem 2.10) in exactly the role it plays here. The contribution of Section 4.5 is not the flow argument but its application to a model recovered from clauses rather than

supplied by a modelling language.

3.3 Symmetry breaking

The families in question are highly symmetric. In PHP_n^m , any permutation of pigeons and any permutation of holes maps the formula to itself, so a search that does not account for symmetry re-derives the same contradiction combinatorially many times.

Static symmetry breaking adds clauses that restrict the search to canonical representatives. Devriendt et al. [8] developed BreakID, which detects row interchangeability and uses stabiliser chains to strengthen the breaking constraints; it won gold in the 2013 and 2016 competitions in combination with contemporary solvers. Anders, Brenner and Rattan [10] extend this in satsuma, detecting row-column and Johnson symmetries in addition, producing stronger reduction while running markedly faster.

This is the most direct competitor to the present work, because it reaches the same instances by a completely different route: rather than recovering the counting constraint, it removes the redundancy that made the resolution proof large. Chapter 10 measures it.

3.4 Pseudo-Boolean solving and cutting planes

The proof-complexity discussion of Section 2.3 identifies cutting planes as the system in which these families are easy. Solvers reasoning natively in that system therefore address the obstacle at its root rather than routing around it.

RoundingSat [9] performs conflict analysis with cut-based reasoning, learning cardinality constraints rather than clauses. Sat4j’s pseudo-Boolean engine uses generalised resolution, and is the solver in the **sgen** result quoted above. Because Theorem 2.7 guarantees polynomial refutations of pigeonhole in this system, a pseudo-Boolean solver is theoretically stronger on precisely the families where the present system performs best. This has not been measured here and is recorded as an outstanding obligation in Chapter 13.

3.5 Structure detection inside modern solvers

The baselines used in this report are CaDiCaL 3.0.1 and Kissat 4.0.4, pinned by commit. It should be noted that CaDiCaL 2.0 [11] introduced clausal congruence closure, detecting AND, XOR and ITE gates encoded in the formula and merging equivalent outputs, alongside clausal sweeping for semantic equivalences. A modern baseline therefore performs some structure recovery of its own, and a fair published comparison must account for this.

3.6 Machine learning and LLM-evolved solvers

Learned components inside CDCL are established but incremental. NeuroBack [12] trains a graph neural network to predict backbone phases offline, requiring no GPU at solve time, and enables Kissat to solve a few per cent more instances on recent competition sets. Bandit-based heuristic selection is now mainstream; the 2025 competition winner is of that lineage. Purely neural solving, in the line of NeuroSAT, has not scaled to competitive performance and the field treats it as complementary to CDCL rather than as a replacement.

The significant recent development is repository-scale code evolution. Yu et al. [14] report SATLUTION, in which LLM agents evolve entire solver repositories — hundreds of files, tens of thousands of lines of C/C++ — under correctness guarantees and distributed runtime feedback, while evolving their own evolution policies. Starting from the 2024 competition codebases, the evolved solvers outperformed the human-designed winners of the 2025 competition.

That work is orthogonal to this one, in a way worth stating precisely. Evolving an implementation of CDCL does not change the proof system it searches in, and Theorem 2.6 is a statement about that system. An evolved CDCL solver should be expected to remain exponential on pigeonhole. The corresponding caution runs the other way: the implementation improvements of Chapter 6 are exactly the kind of gain such a system produces autonomously, so they should not be presented as the durable contribution.

3.7 Summary of positioning

Table 3.1 collects the mapping.

Table 3.1: Each component of the pipeline against its established counterpart.

Component	Established work	Relationship
Cardinality recovery	Biere et al. [7]; Ostrowski et al. [6]	Prior art, and more general: their semantic method handles arbitrary k .
Hidden cardinality	Biere et al. [7]	Prior art; two-product encodings are handled there.
Flow / Hall reasoning	Régin [5]	Prior art; the standard alldifferent propagator.
Symmetry exploitation	BreakID [8], satsuma [10]	Competing route to the same instances; satsuma is broader and faster.
Cutting-plane strength	RoundingSat [9]	Theoretically stronger on these families.
Certified reasoning	Reeves [13]	Same ground as the witness argument.

No row of Table 3.1 is empty, and the report does not pretend otherwise. But the table lists components, and what distinguishes this work is what the components are assembled into.

Every entry in the first column terminates, in the literature, by handing its findings to a search procedure. Detection feeds re-encoding or a pseudo-Boolean engine; symmetry breaking emits clauses for a CDCL solver; the **alldifferent** propagator filters domains inside a constraint solver’s search. In each case the recovered information is an input to a search that still has to run.

The pipeline of Chapter 4 removes the search. The recovered constraints are assembled into an allocation model and that model is *decided* — by a maximum-flow computation whose failure is a refutation of the formula (Theorem 4.6). No solver is invoked on the instances this succeeds on, and the answer arrives with a certificate about the original formula rather than a proof about a transformed one.

Two further observations sharpen the position. The closest prior work [7] names

precisely this gap as open, so the contribution is responsive to a stated question rather than to an invented one. And the certificate of Chapter 5 has no direct counterpart: proof logging for cardinality reasoning [13] certifies derivations inside a solver, whereas what is certified here is a combinatorial refutation over structure recovered from clauses.

The honest summary is therefore neither that the work is unprecedented nor that it recombines existing parts without addition. The parts are established; the composition, the decision procedure it enables and the certificate it admits are not. Chapters 10 and 13 bound what that is worth in measured terms.

CHAPTER 4

The LKK-HSAT pipeline

This chapter develops the six stages formally and proves that a structural refutation is sound. Throughout, F is the input formula and every stage either produces information about F or declines.

4.1 Design constraints

Three requirements shape the design.

Soundness must not depend on discovery.

Search for structure is heuristic and budgeted; whether a candidate is *accepted* must be an exact test. This separation is what allows the budgets to be tuned freely without a soundness argument being revisited.

Noise must not act as counter-evidence.

Adding unrelated clauses to a formula must never turn a non-refutation into a refutation. The acceptance test is therefore purely existential: it asks whether required clauses are present, never whether extra ones are absent.

The cost of failure must be small.

On a formula with no recoverable structure the front-end must decline quickly, because that cost is paid on every instance where the technique does not apply. Section 4.2 and the random controls of Chapter 9 address this.

4.2 Stage 1–2: the gates

The first gate scans the formula once, counting monotone positive and monotone negative clauses and detecting *mirror pairs*: clauses $C \vee y$ and $C \vee \neg y$ sharing a common remainder, which by Definition 2.1 resolve to C and signal the hiding gadget of Equation (2.5). Writing p , n and μ for the counts of monotone positive clauses, monotone negative clauses and mirror pairs, the gate admits the formula when

$$(p > 0 \wedge n > 0) \vee \mu > 0, \quad (4.1)$$

that is, when both directions of a cardinality constraint are represented or a hidden constraint is suggested. The cost is $O(\sum_{C \in F} |C|)$ plus the hashing needed for mirror detection.

The second gate is a budget guard. With $\sigma = p + n + 2\mu$ as a structural signal and $\pi = \sum_{v \in V} |\{C : v \in C\}| \cdot |\{C : \neg v \in C\}|$ as an estimate of resolution work, it admits the formula when

$$\sigma \geq \sigma_{\min} \wedge |\mathcal{C}(F)| \leq m_{\max} \wedge \pi \leq \pi_{\max}. \quad (4.2)$$

Both gates are deterministic functions of the formula; neither consults a file name, a family label or an expected answer.

4.3 Stage 3: bounded resolution recovery

Recovery computes a set $\mathcal{D} \supseteq \mathcal{C}(F)$ closed under resolution up to explicit limits: a maximum derivation depth δ , a maximum clause length λ , a cap on derived clauses, a memory bound, a wall-clock budget, and a restriction that pivots must occur at most ρ times in the original formula. The last restriction is what keeps the closure small: it confines derivation to variables that look like encoding auxiliaries rather than problem variables.

Proposition 4.1 (Recovery soundness). *Every $D \in \mathcal{D}$ satisfies $F \models D$, regardless of which budgets terminate the computation.*

Proof. Immediate from Lemma 2.2: $\mathcal{D}$ is generated from $\mathcal{C}(F)$ by resolution alone, and the budgets only stop the process earlier. Termination therefore affects completeness, never soundness. $\square$

Proposition 4.1 carries the entire soundness burden of the discovery half of the pipeline. If recovery aborts on a budget, the registry simply sees fewer clauses and finds fewer families.

4.4 Stage 4: the registry

The registry turns clauses into an allocation model. It looks for two kinds of object.

Definition 4.2 (Box). A *box* is a pair (G, r) with $G \subseteq V$, $0 < r < |G|$, such that $\mathcal{F}_{=r}(G) \subseteq \mathcal{D}$, in the sense of Definition 2.4. A box asserts that exactly r variables of G are true.

Definition 4.3 (Resource). A *resource* is a pair (H, c) with $H \subseteq V$ such that $\text{AtMost}_c(H) \subseteq \mathcal{D}$, following Equation (2.1). A resource asserts that at most c variables of H are true.

Candidate boxes are proposed by pairing a monotone positive clause P with a monotone negative clause N whose variable sets share exactly two elements, taking $G = \text{var}(P) \cup \text{var}(N)$ and $r = |N| - 1$; the exhaustive membership test of Definition 4.2 then decides the candidate. Chapter 6 analyses the cost of this search, which is where the Phase 4C contribution lies.

Boxes are selected greedily in order of decreasing size, subject to being pairwise variable-disjoint — an assignment variable can belong to only one box in the allocation reading. Resources are selected the same way. The registry reports success only when every variable covered by a selected box is also covered by exactly one selected resource; otherwise it reports UNKNOWN and the formula goes to the fallback.

Remark 4.4. The disjointness requirement discards proven constraints. This is a deliberate loss of strength in exchange for a model whose flow reading is unambiguous, and it is one reason the technique is incomplete even where structure exists.

4.5 Stage 5: the flow test

Let $\mathcal{B} = \{(G_1, r_1), \dots, (G_b, r_b)\}$ and $\mathcal{R} = \{(H_1, c_1), \dots, (H_q, c_q)\}$ be the selected boxes and resources, with every variable of every G_i lying in exactly one H_j . Write $\phi(v)$ for the index of the resource containing v , and require that ϕ is injective on each G_i , so a box never places two of its variables in one resource.

Definition 4.5 (Allocation network). The network $N(\mathcal{B}, \mathcal{R})$ has nodes $\{s\} \cup \{u_1, \dots, u_b\} \cup \{w_1, \dots, w_q\} \cup \{t\}$ and arcs

$$s \rightarrow u_i \text{ with capacity } r_i, \quad u_i \rightarrow w_{\phi(v)} \text{ with capacity 1 for each } v \in G_i, \quad w_j \rightarrow t \text{ with capacity } c_j.$$

Its *total demand* is $\Delta = \sum_{i=1}^b r_i$.

Theorem 4.6 (Structural refutation). *Let $\mathcal{B}$ and $\mathcal{R}$ be as above with every box and every resource entailed by F . If the maximum s - t flow in $N(\mathcal{B}, \mathcal{R})$ is strictly less than Δ , then F is unsatisfiable.*

Proof. Suppose α satisfies F . Since each box (G_i, r_i) is entailed, α makes exactly r_i variables of G_i true; let $T_i \subseteq G_i$ be that set, so $|T_i| = r_i$. Define an arc assignment f by sending one unit along $u_i \rightarrow w_{\phi(v)}$ for each $v \in T_i$, r_i units along $s \rightarrow u_i$, and $\sum_i |\{v \in T_i : \phi(v) = j\}|$ units along $w_j \rightarrow t$.

We check f is a feasible flow. Arc capacities $s \rightarrow u_i$ are met with equality. Each arc $u_i \rightarrow w_{\phi(v)}$ carries at most one unit because ϕ is injective on G_i , so distinct $v \in T_i$ use distinct arcs. For the sink arcs, the variables routed to w_j are exactly those of $\bigcup_i T_i$ lying in H_j ; since the resource (H_j, c_j) is entailed and α satisfies F , at most c_j variables of H_j are true under α , hence at most c_j units arrive at w_j . Conservation at u_i and w_j holds by construction.

The value of f is $\sum_i r_i = \Delta$. Therefore a satisfying assignment yields a flow of value Δ , and contrapositively a maximum flow below Δ implies no satisfying assignment exists. $\square$

Corollary 4.7 (Witness). *When the maximum flow is below Δ , Theorem 2.9 yields a cut of capacity equal to that flow. The set of boxes on the source side constitutes a Hall violation in the sense of Theorem 2.10: their combined demand exceeds the capacity reachable from them.*

The cut is recorded in the witness, so a checker can confirm the refutation without repeating the flow computation: it re-derives the clause database from the parents, re-verifies each family by Definitions 4.2–4.3, rebuilds the network and confirms the reported cut separates source from sink with the stated capacity.

4.6 Stage 6: witness or fallback

A refutation writes a witness containing the recovered clause database with each clause’s resolution parents and pivot, the accepted boxes and resources with the identifiers of their evidence clauses, the variable-to-resource map, and the flow figures with the min-cut. Anything else — gates declining, recovery aborting, the registry reporting UNKNOWN, or the flow proving feasible — hands the *unchanged* formula to a locked CDCL solver. The formula given to the fallback is never modified, which is what makes the fallback’s answer a statement about the original instance.

Remark 4.8 (Feasibility is not satisfiability). A feasible flow does not imply F is satisfiable. The allocation model is a relaxation: it captures the recovered cardinality constraints and nothing else, so it can be feasible while the formula is unsatisfiable for unrelated reasons. The pipeline is therefore a one-sided decision procedure — it can refute, never affirm — and every satisfiable instance in Chapter 9 is answered by the fallback.

CHAPTER 5

Certification

A structural refutation is only as trustworthy as the argument that it is correct. This chapter specifies the certificate the engine emits and the procedure that checks it, and states precisely what a successful check establishes.

5.1 What a certificate must establish

By Theorem 4.6, a refutation rests on three claims: that every clause used is entailed by the original formula; that each accepted box and resource really is the constraint it purports to be; and that the allocation network built from them has maximum flow below total demand. A certificate must allow each to be checked without trusting the engine that produced it, and without repeating its search.

Note what is *not* required. The checker need not re-run the recovery, which is the expensive and heuristic part. It need only verify the derivation that recovery claims to have found, which is a linear pass.

5.2 The witness format

A witness is a self-contained record with five components.

Formula identity.

The SHA-256 of the original CNF, its variable count and its clause count. A checker refuses a witness that does not match the formula it is given, so a certificate cannot be silently paired with the wrong instance.

Clause database.

Every clause of $\mathcal{D}$, each with an identifier, its literals, its derivation depth, a flag marking it original or derived, and for derived clauses the identifiers of its two parents together with the pivot literal.

Boxes.

For each accepted box: its variables, its demand r , and the identifiers of the database clauses constituting $\mathcal{F}_{=r}(G)$.

Resources.

For each accepted resource: its variables, its capacity c , and the identifiers of the clauses constituting $\text{AtMost}_c(H)$.

Flow.

The network size, the total demand Δ , the maximum flow value, and the minimum cut: the boxes and resources on the source side, with the three components of the cut capacity.

Recording parents and pivots rather than merely the derived clauses is what makes the entailment claim checkable rather than assertable.

5.3 The checking procedure

The checker takes a witness and the original CNF and performs four passes.

Pass 1: replay. Process the database in identifier order. A clause marked original must be present in $\mathcal{C}(F)$; a derived clause must equal $\text{Res}_\ell(C_a, C_b)$ for its recorded parents and pivot, where both parents have smaller identifiers. The ordering condition rules out circular justification. If every record passes, Lemma 2.2 gives $F \models D$ for all $D \in \mathcal{D}$.

Pass 2: families. For each box (G, r) , regenerate $\mathcal{F}_{=r}(G)$ from Definition 2.4 — the checker computes the required set itself rather than trusting the witness’s list — and confirm every member appears in the replayed database. Likewise for each resource. This is where a fabricated constraint would be caught: a box claiming a demand its clauses do not support fails here.

Pass 3: model well-formedness. Confirm that the boxes are pairwise variable-disjoint, that every box variable lies in exactly one resource, and that no box has two variables in the same resource. These are the hypotheses of Theorem 4.6; without them the flow reading is not justified.

Pass 4: the cut. Rebuild the network of Definition 4.5 and confirm that the reported cut separates source from sink and that its capacity equals the reported maximum flow and is strictly below Δ . By Theorem 2.9 the exhibited cut bounds the maximum flow from above, so this direction alone suffices: the checker does not need to compute a maximum flow, only to validate a certificate of its being small.

Proposition 5.1 (Soundness of checking). *If a witness passes all four passes against a formula F , then F is unsatisfiable.*

Proof. Pass 1 establishes $F \models D$ for every database clause. Pass 2 establishes that each box (G, r) has $\mathcal{F}_{=r}(G) \subseteq \mathcal{D}$, hence F entails the constraint “exactly r of G ”, and similarly for resources. Pass 3 supplies the hypotheses of Theorem 4.6. Pass 4 exhibits a cut of capacity below Δ , which by Theorem 2.9 bounds the maximum flow below Δ . Theorem 4.6 then yields unsatisfiability. $\square$

Remark 5.2 (Independence). The checker used in this work is a separate Python implementation sharing no code with the C++ engine. It reads the witness and the CNF and nothing else — not the engine’s configuration, not its logs, not its budgets. An error in the engine’s search cannot make an unsound witness pass, because the checker recomputes the required families itself rather than accepting the engine’s account of them.

5.4 What a passing check does not establish

Three limits deserve statement.

A witness certifies unsatisfiability, not the absence of a shorter proof, and carries no information about satisfiable instances: when the flow is feasible no witness is emitted, and the fallback solver’s answer stands on its own.

A passing check says nothing about the recovery budgets. Different budgets would produce a different database and possibly a different set of boxes; the certificate concerns the run that produced it.

Finally, the checker validates the mathematical content, not the engineering. It does not detect that the engine spent longer than necessary, or that it declined an instance it could have decided. Those are the concerns of the correctness campaign of Section 8.2 and of the accounting analysis of Chapter 6, which exist precisely because certification alone does not cover them.

5.5 Relation to proof logging in the literature

Proof certificates for clausal reasoning are standard: DRAT is required output in the SAT competition, and checked proofs are the norm for unsatisfiability claims. Certification for *cardinality* reasoning is more recent and is the subject of active work; Reeves [13] develops proof logging for cardinality constraints in exactly this spirit, and pseudo-Boolean solvers such as RoundingSat [9] emit cutting-plane proofs.

The certificate described here is narrower than those: it certifies one specific argument form — a Hall violation over recovered cardinality constraints — rather than an arbitrary derivation. Its advantage is that it is about the original formula. Preprocessing techniques that transform the instance, symmetry breaking among them, produce proofs about the transformed instance, so trusting them as statements about the original requires trusting the transformation as well as the proof. That difference is the one qualitative advantage Chapter 10 is able to claim.

CHAPTER 6

The cost of the registry search

The registry of Section 4.4 is the expensive stage, and the budget governing it is the parameter that decides whether an instance is solved structurally or handed to the fallback. This chapter analyses that cost, shows that the natural implementation mischarges its own budget in two opposite directions, and quantifies the effect of correcting it.

6.1 The candidate search

Let P and N be the monotone positive and monotone negative clauses of the recovered database $\mathcal{D}$. A box candidate arises from a pair $(p, n) \in P \times N$ whose variable sets intersect in exactly two elements. The direct implementation enumerates the product:

$$T_{\text{scan}} = |P| \cdot |N|. \quad (6.1)$$

Almost all of that work is discarded. A pair qualifies only when $|\text{var}(p) \cap \text{var}(n)| = 2$, and on the benchmark families the qualifying fraction is small, so Equation (6.1) is dominated by rejections.

The alternative is to index the negatives by variable pair. Let

$$I(\{a, b\}) = \{n \in N : \{a, b\} \subseteq \text{var}(n)\}, \quad (6.2)$$

built in $\sum_{n \in N} \binom{|\text{var}(n)|}{2}$ steps, which is linear in $|N|$ for bounded clause length. Every qualifying partner of p appears in $I(\{a, b\})$ for the unique shared pair $\{a, b\}$, so enumerating $\bigcup_{\{a, b\} \subseteq \text{var}(p)} I(\{a, b\})$ visits every qualifying pair and no pair that shares fewer than two variables. The cost becomes

$$T_{\text{index}} = \sum_{p \in P} \binom{|\text{var}(p)|}{2} \cdot \bar{\beta}, \quad (6.3)$$

where $\bar{\beta}$ is the mean bucket size. A partner sharing three or more variables occurs in several buckets and is filtered by a visit stamp, so it is examined once, keeping the count of examined pairs faithful to the scan it replaces.

Proposition 6.1. *With an unbounded budget, the indexed search accepts exactly the same set of box candidates as the product scan.*

Proof. Both enumerate a superset of the qualifying pairs and apply the identical acceptance test of Definition 4.2. A pair with $|\text{var}(p) \cap \text{var}(n)| = 2$ is reached by the scan directly and by the index through its unique shared pair; a pair sharing fewer than two variables occurs in no bucket and is rejected by the scan's test. The candidate dictionary is keyed by (G, r) and the selection sorts it, so discovery order does not affect the outcome. $\square$

6.2 Two accounting defects

Both stages of the registry consume a shared budget of *checks*. The budget's purpose is to bound effort, so a check should correspond to a roughly constant amount of work. Two places violated this.

6.2.1 Overcharging: the family test

Testing a box candidate (G, r) requires deciding $\mathcal{F}_{=r}(G) \subseteq \mathcal{D}$. The natural implementation materialises the family and charges its size, Equation (2.3), before testing any member:

$$\text{charge} = \binom{|G|}{r+1} + \binom{|G|}{|G|-r+1}, \quad \text{work performed until the first miss} = 1 \text{ in the common case.}$$

For $|G| = 11$, $r = 2$ this charges 176 checks (Example 2.5) to learn, typically, that the very first clause is absent. The budget retires having examined a small number of candidates, none of them decided cheaply. Charging per membership test and stopping at the first miss gives the identical verdict for

$$\text{charge} = \min \{ i : \text{the } i\text{-th required clause is absent} \} \quad (6.4)$$

checks on a rejected candidate, and the full family only on an accepted one. Memoising decided keys (G, r) — accepted *and* rejected — removes the repeated cost of candidates reachable from many pairs.

6.2.2 Undercharging: the clique growth

Resources are assembled by growing cliques in the graph whose edges are the monotone negative clauses of a given size s . Testing whether a vertex x extends a clique K requires checking that every s -subset of $K \cup \{x\}$ is an edge. The natural implementation charges one check per candidate while performing

$$\binom{|K|+1}{s} \quad (6.5)$$

edge lookups. The budget therefore appears almost untouched while the search spends seconds. Two corrections apply. First, K is already a clique, so every subset omitting x is known to be an edge and only those containing it can fail; the necessary and sufficient test is over

$$\binom{|K|}{s-1} \quad (6.6)$$

subsets, which for $s = 2$ and $|K| = 24$ is 24 lookups rather than 300. Second, each lookup is charged, restoring the budget as a bound on effort.

A third inefficiency is not an accounting error but a complexity one: the candidate pool and the occupied-resource set are properties of K , and recomputing them at every growth step makes reaching a clique of size k cost $O(k^2)$ sweeps. Maintaining them incrementally — one set intersection per accepted vertex — reduces this to $O(k)$.

Remark 6.2 (Why a profiler would not have found these). Both defects are mismatches between what the counter claims and what the code does. A profiler reports that time is spent in the registry, which was already known; it cannot report that the budget is being charged for work not performed. The tests described in Section 8.6 make the invariant checkable by instrumenting the clause index and comparing lookups against the charge.

6.3 Refitting the budget

Charging previously unbilled work changes what a check costs, so the constant calibrated against the old accounting no longer denotes the same quantity of effort. Choosing a

replacement by observing the benchmark respond would be tuning on the test set, which the methodology of Chapter 8 forbids. The value was instead fitted by an F-race over candidate budgets on 40 generated calibration instances whose seeds are disjoint from every benchmark range, scored by mean PAR-2. Table 6.1 reports the race.

Table 6.1: Budget selection by F-race on the calibration split. Configurations falling behind the leader are eliminated and do not complete the field, which is why the evaluation counts differ. The selected value is 300 000.

Budget	Evaluations	Mean PAR-2 (ms)	Outcome
100 000	18	eliminated	raced out
200 000	40	1,571.9	
300 000	40	1,526.9	selected
500 000	40	1,528.8	
1 000 000	40	1,532.9	

Remark 6.3 (A calibration set has to look like the evaluation set). The first race reported every candidate budget as equivalent. The cause was that the calibration families omitted the scales at which the budget actually binds, so no instance in the race was budget-limited. Sharing families and scales with the benchmark — under disjoint seeds, and without consulting any benchmark result — is not leakage; it is the condition under which a calibration set measures anything at all.

6.4 Effect

Table 9.2 in Chapter 9 gives the per-stage cost after these corrections. The qualitative effect is that instances previously abandoned by the registry are now decided: the budget buys examinations of candidates rather than enumerations of clauses never tested, and the clique growth no longer consumes unmetered time. The quantitative effect is Table 9.1, whose extreme row is *exact- r* at scale 11 — unsolved by any locked baseline within the benchmark budget, decided structurally afterwards.

CHAPTER 7

Transferring recovered structure: a negative result

The pipeline of Chapter 4 either refutes a formula structurally or hands it, unchanged, to a CDCL solver. A natural question is whether the intermediate case can be exploited: when the registry finds real structure but the flow test does not refute, can that structure be handed to the solver as clauses, so that CDCL searches a smaller space?

This chapter reports the experiment that asked the question. Its answer is negative, and the negative answer is more informative than a marginal positive one would have been.

7.1 What may be transferred

Soundness sets the boundary. A clause may be given to the fallback solver only if the original formula entails it, since the solver’s answer must remain an answer about the original instance. Three sources satisfy this.

Resolvents.

Any clause of the recovered database $\mathcal{D}$ is entailed by Proposition 4.1. Those not already in $\mathcal{C}(F)$ are new information for the solver, though not new logical content.

Family clauses.

The members of $\mathcal{F}_{=r}(G)$ for an accepted box, and of $\text{AtMost}_c(H)$ for an accepted resource. These are entailed by the acceptance test itself, since acceptance requires them to be present in $\mathcal{D}$.

Conditional capacity conflicts.

If assuming a set of literals $L = \{x_1, \dots, x_k\}$ makes the allocation model infeasible, then by the argument of Theorem 4.6 no satisfying assignment extends L , so $F \models \bigvee_i \neg x_i$.

The third source is the interesting one: it manufactures clauses that are not present anywhere in $\mathcal{D}$, derived from the geometry of the flow network rather than from resolution.

7.2 Independent verification

Because a wrong lemma would corrupt the solver’s answer silently, every candidate was verified by two independent routes before use.

The first is a replay. The witness carries the recovered database with each clause’s resolution parents and pivot, so a checker can re-derive every clause from the original CNF alone, confirming Definition 2.1 at each step. This establishes entailment constructively.

The second is a satisfiability check. For a candidate clause D , the checker constructs $F \wedge \neg D$ — that is, F together with a unit clause for the negation of each literal of D — and runs a locked solver that took no part in generating the lemma. Entailment holds exactly when this is unsatisfiable:

$$F \models D \iff F \wedge \bigwedge_{\ell \in D} \neg \ell \text{ is unsatisfiable.} \quad (7.1)$$

A candidate that fails this check is a counterexample and stops the campaign; one that exhausts its verification budget is rejected rather than trusted.

7.3 Results of the verification campaign

Table 7.1 reports the campaign. Every one of the 3344 candidates passed both checks, at a total independent verification cost of 65.1 s.

Table 7.1: Lemma generation and independent verification, Phase 4B. Every candidate was checked by resolution replay and by an implication test on a locked solver that played no part in its generation.

Quantity	Count	Note
Candidates generated	3344	across the eligible instances
from exact- r families	1262	members of $\mathcal{F}_{=r}(G)$
from capacity families	1386	members of $\text{AtMost}_c(H)$
from bounded resolution	696	derived, not present in F
Verified and accepted	3344	both checks passed
Failed implication check	0	no counterexample found
Unverified within budget	0	none rejected on time
Total verification time	65.1 s	

7.4 The negative result

Injecting the verified lemmas into the fallback solver improved end-to-end runtime on *zero* instances. The reasons differ by family and are worth separating, because each rules out a different hypothesis.

On directly encoded families the lemmas are not new. For pigeonhole, exact- r and capacity-2, the family clauses of an accepted box are exactly the clauses the encoder emitted. Recovering them and handing them back gives the solver nothing it did not already have; the transfer is a no-op that costs generation time.

On hidden families the lemmas are new but do not help enough. Where the constraint is buried behind auxiliary variables, the recovered family clauses genuinely are absent from F . Supplying them shortens the proof the solver must find — but the shortened proof is still a resolution proof of a pigeonhole-like statement, and Theorem 2.6 still applies to it. The lemmas move the constant, not the exponent.

Conditional conflicts are degenerate where they exist. The conditional source requires an allocation model that is feasible unconditioned and infeasible after assuming some literals. On the families studied, whenever a usable sub-model existed it was *already* infeasible unconditioned — that is, the structure refuted the formula outright, which the pipeline of Chapter 4 reports directly. An unconditional refutation expressed as a clause is degenerate: it entails every unit clause, so emitting it as a lemma is both useless and misleading. The generator therefore returns no lemma in that case and records the refutation as a finding.

7.5 What the experiment did reveal

The campaign produced one substantive discovery, and it came from the machinery built for the lemma experiment rather than from the lemmas.

To find conditional conflicts, the generator harvests structure under a wider budget than the accepted registry uses and retains partial results where the accepted registry reports UNKNOWN. On 14 instances that partial structure was already sufficient to refute the formula by Theorem 4.6, each confirmed independently and never contradicting a locked solver.

That observation is what motivated Chapter 6. If a wider budget over the same search decides instances the accepted budget abandons, then the interesting question is not how to transfer structure to the solver but why the budget was exhausted before finding it. The answer turned out to be mischarging rather than genuine cost, and correcting it made the wider budget unnecessary: the instances in question are now decided within the fitted budget by the engine of Chapter 9.

Remark 7.1 (Why the negative result was worth the effort). A lemma-transfer layer that produced a small speedup would have been reported as a success and carried forward as complexity. Establishing that it produces none, under a verification regime strict enough that a wrong lemma would have stopped the campaign, removed a component from the architecture and redirected the work to the stage that mattered. The verification machinery itself was retained: it is the same replay checker that validates the witnesses of Section 4.5.

CHAPTER 8

Methodology

8.1 Phase discipline

Development proceeded in sealed phases. When a phase is accepted, its artifact directory is hashed and thereafter treated as immutable: every later campaign computes the digest of each predecessor before and after its own run and aborts if either differs. A phase may therefore be compared against, but never silently amended. Failed and superseded runs are retained rather than deleted, so the record contains the campaigns that exposed defects alongside those that established results.

Table 8.1: Development phases. Each is a sealed artifact directory.

Phase	Subject	Content
1	Locked baseline	CaDiCaL 3.0.1 and Kissat 4.0.4 pinned by commit, built in a digest-pinned image.
2	Structural engine	Bounded recovery, registries, FlowBoxTest, witness format and independent checker.
3	Performance campaign	Fourteen generated instances over seven families, three seeds, interleaved order.
4A	Native port	The structural path in C++ with in-process CaDiCaL, removing a process launch and a reparse.
4B	Fallback engines	External Kissat as an alternative fallback; verified lemma transfer, each lemma independently certified.
4C	Indexed registry	The search of Chapter 6, with the shared core and its drift test.
5	Public benchmarks	1 192 instances acquired and verified; campaign prepared, not run.

8.2 The correctness gate

No performance figure in this report comes from a campaign that had not first passed a correctness gate. A campaign failing the gate terminates before it measures anything, and its partial artifacts are preserved.

The gate has six components.

Immutable predecessors.

Tree digests of every accepted phase directory, compared before and after.

Flow cross-checks.

5 000 freshly generated capacity instances are decided both by max-flow and by exhaustive enumeration over all subsets, in the Python reference and in the C++ engine. Any disagreement stops the campaign.

Answer agreement.

For every instance, the definite answers of the Python reference, the native engine and both locked solvers must coincide.

Witness verification.

Every structural refutation is re-checked by an independent Python checker reading only the witness and the original CNF.

Core drift.

The shared structural core is differential-tested field by field against the sealed Phase 4A binary, so the claim that the accepted semantics are unchanged is enforced rather than asserted. The Phase 4C search is held to a weaker but explicit contract: it may decide *more*, never differently.

Accounting invariants.

Tests instrument the clause index and assert that every lookup is charged to the budget, guarding the defect class of Chapter 6.

8.2.1 An incident the gate caught

The value of the gate is best shown by a failure. A campaign intended to produce the final figures of this report stopped at the correctness stage, reporting that the C++ engine and the Python reference disagreed on two capacity-2 instances: the engine decided them structurally, the reference did not.

The engines were in fact identical. Both consumed exactly 284,856 checks on the first instance and produced the same boxes and resources. The divergence was in the harness: after the budget was refitted (Section 6.3), the C++ engine used the new value from its own default while the Python reference still read the old value from the Phase 3 configuration file. The two were being run on the same instance under different budgets.

The repair was to pass the fitted budget explicitly to both, so neither depends on a compiled-in or inherited default. Had the gate not been in place, this report would have contained a false claim of cross-implementation agreement. The stopped campaign is retained.

8.3 Benchmark families

The generated corpus comprises seven families, chosen to separate the effects under study.

Pigeonhole.

Direct encoding of PHP_n^{n+1} as in Equation (2.4). Unsatisfiable, exponential for resolution.

Exact- r allocation.

Boxes of demand 2 over a shared pool of unit resources; unsatisfiable by a counting argument, with families of size given by Equation (2.3).

Overlapping Hall violation.

A violating core plus an equally sized satisfiable component, so the contradiction is local rather than visible from total demand against total capacity.

Capacity-2.

Resources of capacity 2, exercising the clique growth of Section 4.4 at $s = 3$.

Hidden pigeonhole.

Pigeonhole with every clause passed through the hiding gadget of Equation (2.5), optionally with planted noise, so no cardinality clause is present syntactically.

Satisfiable capacity control.

A tight but satisfiable allocation. The structure is genuine and recoverable, yet the flow is feasible, so the instance must fall through to the fallback. It measures the cost of finding structure that does not help.

Random 3-SAT.

Uniform and planted instances at the threshold ratio, with no recoverable structure. These measure the overhead paid where the technique does not apply.

The last two families exist to make the evaluation falsifiable. A structural front-end can always be made to look good by measuring it only on instances built for it.

8.4 Measurement protocol

Each instance is run under every mode, with the order of modes rotated per instance and per seed so that no mode systematically occupies a warm or cold cache position. Three seeds are used. The timeout is 5 s and results are aggregated by PAR-2, that is, the runtime for a solved instance and twice the timeout otherwise. The primary measure is process-level end-to-end wall time, so the structural front-end is charged for everything it does, including on instances where it decides nothing.

Solvers are pinned by commit and recorded by SHA-256 in each run’s metadata: CaDiCaL 3.0.1 (c607304) and Kissat 4.0.4 (8af8e56), both compiled `-O3 -DNDEBUG` inside a digest-pinned base image. The sealed Phase 4A binary is rebuilt from its unchanged source for every campaign so that the comparison is against the engine that produced the accepted results, not against a remembered figure.

8.5 The prepared public corpus

A corpus of 1 192 public instances has been acquired and verified for the campaign that Chapter 14 describes as outstanding. Table 8.2 gives its composition by provenance domain. Every instance carries its source, licence and SHA-256 in a manifest; no formula was generated by this project, and the only transformation applied is decompression, with both digests recorded.

Table 8.2: Composition of the prepared public corpus by provenance domain. Sources are SATLIB and the SAT Competition 2024 main track (Zenodo, 10.5281/zenodo.15095752, CC-BY-4.0, verified against the published MD5).

Provenance domain	Instances	Share
random control	402	33.7%
matching capacity	313	26.3%
competition main	250	21.0%
crafted structured	122	10.2%
crafted parity	51	4.3%
planning scheduling	29	2.4%
eda formal verification	13	1.1%
eda circuit fault	12	1.0%
total	1192	100.0%

8.6 Accounting invariant tests

The defects of Chapter 6 are mismatches between a counter and the work it purports to measure, which no profiler detects. They are guarded by tests that wrap the clause index in a counting proxy and compare the observed lookups against the charge. Two invariants are asserted: every lookup is charged, so the budget can never be unaware of work performed; and the charge stays within a small factor of the lookups on structured instances, so the budget cannot retire having paid for work it did not do. A third test documents that the accepted registry overcharges, so that if this ever ceases to hold the rationale for Phase 4C is revisited rather than silently invalidated.

CHAPTER 9

Results

All figures in this chapter come from campaign 20260817Tphase4cfinal04Z: 14 instances, three seeds, interleaved mode order, 5 s timeout, PAR-2 aggregation. The campaign passed the gate of Section 8.2 with 0 flow mismatches over 5 000 cross-checks and 0 failures over 46 correctness cases.

9.1 Headline comparison

Table 9.1 compares the Phase 4C engine against the sealed Phase 4A engine and both locked solvers. Instances marked † are those the sealed engine left to the fallback — or could not finish at all — and that Phase 4C decides structurally.

Table 9.1: Mean PAR-2 over three seeds, in milliseconds. A structural verdict of UNSAT means the instance was refuted by Theorem 4.6 without invoking a CDCL solver. Instances marked † are newly decided structurally; *s* denotes scale and ⁺ a noisy variant.

Instance	Structural verdict		PAR-2 (ms)				
	4A	4C	4A	4C	CaDiCaL	Kissat	4C/4A
001 pigeonhole <i>s</i> 8	UNSAT	UNSAT	36.9	20.5	119.6	68.7	1.80
002 pigeonhole <i>s</i> 10	UNSAT	UNSAT	102.0	18.6	4,619.4	502.6	5.50
003 exact- <i>r</i> <i>s</i> 9	UNSAT	UNSAT	46.1	19.3	217.8	184.9	2.40
004 exact- <i>r</i> <i>s</i> 11†	timeout	UNSAT	10,000.0	18.2	10,000.0	10,000.0	548.78
005 Hall <i>s</i> 8†	UNKNOWN	UNSAT	787.9	30.6	520.1	151.9	25.78
006 Hall <i>s</i> 9†	UNKNOWN	UNSAT	4,118.0	36.0	4,134.5	368.8	114.52
007 capacity-2 <i>s</i> 5	UNSAT	UNSAT	269.1	84.4	1,189.5	302.6	3.19
008 capacity-2 <i>s</i> 5 ⁺	UNSAT	UNSAT	402.9	101.3	1,591.3	268.7	3.98
009 hidden pigeonhole <i>s</i> 9	UNSAT	UNSAT	67.8	29.6	486.1	134.7	2.29
010 hidden pigeonhole <i>s</i> 9 ⁺ †	UNKNOWN	UNSAT	553.9	36.4	620.4	169.7	15.20
011 SAT control <i>s</i> 24	UNKNOWN	UNKNOWN	85.2	119.0	118.1	35.9	0.72
012 random 3-SAT <i>s</i> 150	UNKNOWN	UNKNOWN	68.1	68.5	118.5	100.8	0.99
013 random 3-SAT <i>s</i> 400	UNKNOWN	UNKNOWN	821.0	838.5	871.7	1,005.6	0.98
014 random 3-SAT <i>s</i> 800	UNKNOWN	UNKNOWN	972.0	905.1	887.0	151.4	1.07

Four observations follow.

The extreme case. Exact-*r* at scale 11 is not solved by either locked baseline within the benchmark budget, and the sealed engine likewise fails to finish; all three carry the PAR-2 timeout penalty. Phase 4C decides it structurally in tens of milliseconds, a ratio of 549× against the sealed engine. The gap is not an implementation win. It is the difference between searching for a resolution proof that Theorem 2.6 guarantees must be exponential and computing a maximum flow.

The structural families. Pigeonhole, Hall and hidden pigeonhole all improve, and 12 of 14 instances beat both locked solvers by mean PAR-2. The median improvement

over the sealed engine is $2.79\times$.

The satisfiable control regresses. The satisfiable capacity control is slower under Phase 4C than under the sealed engine. This is the honest price of the accounting of Chapter 6: the budget is now spent on real edge lookups rather than on cheap rejections, so the front-end works harder before giving up. Because the instance is satisfiable, the recovered structure cannot help — the flow is feasible and the formula still goes to the fallback. The regression is consistent across seeds and is not hidden.

The random controls are flat. On uniform and planted random 3-SAT the ratio is close to unity. The front-end declines quickly and the cost is dominated by the fallback solver, which is the intended behaviour on instances with no recoverable structure.

9.2 Where the time goes

Table 9.2 decomposes the Phase 4C engine’s cost by stage on the first seed. It shows that recovery is cheap everywhere; that the registry is the expensive stage, as Chapter 6 predicts; and that on the structural instances the flow computation itself is negligible.

Table 9.2: Per-stage cost of the Phase 4C engine, seed 1, in milliseconds. The final column is the number of family checks consumed against a budget of 300 000. A fallback time of zero means the instance was decided structurally.

Instance	Recovery	Registry	Flow	Fallback	Total	Checks
001 pigeonhole s_8	0.13	2.01	0.014	0.0	6.4	11124
002 pigeonhole s_{10}	0.20	4.89	0.018	0.0	7.8	36775
003 exact- r s_9	0.22	1.61	0.015	0.0	5.4	5955
004 exact- r s_{11}	0.41	2.81	0.013	0.0	6.5	16731
005 Hall s_8	0.29	5.48	0.080	0.0	9.9	35405
006 Hall s_9	0.44	10.02	0.096	0.0	15.3	63154
007 capacity-2 s_5	0.30	41.98	0.014	0.0	46.7	284856
008 capacity-2 s_5^+	0.68	42.90	0.017	0.0	48.4	284858
009 hidden pigeonhole s_9	0.96	6.63	0.014	0.0	10.6	22599
010 hidden pigeonhole s_9^+	1.75	8.27	0.019	0.0	15.8	22599
011 SAT control s_{24}	3.35	50.15	0.000	1.8	72.2	300001
012 random 3-SAT s_{150}	0.30	0.07	0.000	34.9	40.2	15
013 random 3-SAT s_{400}	0.64	0.14	0.000	796.3	801.9	11
014 random 3-SAT s_{800}	1.67	0.38	0.000	768.7	778.9	20

Two patterns are worth drawing out. On the random controls the registry cost collapses to a fraction of a millisecond and the check count to a few dozen: the indexed search of Section 4.4 rejects the formula almost immediately instead of grinding through a product of monotone clauses. On the capacity-2 family the registry remains the dominant term, consuming a large share of the budget in clique growth, which is where the remaining headroom lies.

9.3 Correctness campaign

Table A.1 in Appendix A lists every correctness case with the verdicts of the sealed engine, the Phase 4C engine, the Python reference and both locked solvers. All 46 cases agree.

The campaign additionally verified every structural refutation through the independent checker. A witness records the recovered clause database with each clause's resolution parents and pivot, the accepted families with the identifiers of their evidence clauses, and the min-cut of Corollary 4.7; the checker re-derives the database from the original CNF, re-verifies each family against Definitions 4.2–4.3, and confirms the cut. No witness was rejected.

9.4 Reading the numbers correctly

Two cautions apply to Table 9.1 and are developed in Chapter 10.

The comparison against CaDiCaL and Kissat is a comparison against *plain* CDCL. It says nothing about what those solvers achieve when combined with a preprocessor aimed at these families, and Chapter 10 shows that such a combination reaches the same instances at a comparable scale.

The instances are generated by this project. They were designed to exercise the mechanism under study, including two families designed to make it look bad, but they remain synthetic. The public campaign that would test transfer has been prepared and not run.

CHAPTER 10

Comparison with competing techniques

Chapter 9 measured the system against plain CDCL. This chapter measures it against the technique that most directly addresses the same instances, and discusses two others that were not measured.

10.1 Static symmetry breaking

BreakID [8] was built from source on the same machine and run as a preprocessor ahead of the locked CaDiCaL, with its own runtime charged to the pipeline exactly as the structural front-end’s runtime is charged to LKK-HSAT. Table 10.1 reports single runs at a 20 s timeout; these are indicative rather than a formal campaign, and are not aggregated over seeds.

Table 10.1: LKK-HSAT against static symmetry breaking, in milliseconds. The BreakID column is the total of symmetry detection, constraint generation and the subsequent CaDiCaL solve. Single runs, 20 s timeout.

Instance	LKK-HSAT	BreakID + CaDiCaL	CaDiCaL	Kissat	Faster
004 exact- <i>r</i> <i>s</i> 11	16.1	38.7	20,003.0	20,002.2	LKK
002 pigeonhole <i>s</i> 10	15.2	25.9	3,750.9	371.3	LKK
006 Hall <i>s</i> 9	25.3	38.5	4,571.3	406.9	LKK
007 capacity-2 <i>s</i> 5	60.1	28.8	1,111.5	286.7	BreakID
008 capacity-2 <i>s</i> 5 ⁺	58.6	30.9	1,318.8	202.0	BreakID
009 hidden pigeonhole <i>s</i> 9	18.3	205.9	355.3	112.2	LKK
010 hidden pigeonhole <i>s</i> 9 ⁺	23.0	232.9	368.5	110.3	LKK
011 SAT control <i>s</i> 24	85.2	285.3	41.9	9.7	LKK
014 random 3-SAT <i>s</i> 800	709.9	972.7	1,029.9	121.8	LKK

The result is unambiguous and constrains the claims of this report.

Symmetry breaking reaches the extreme case. On exact-*r* at scale 11, where both locked solvers time out at 20 s, BreakID plus CaDiCaL finishes in under 40 ms. The qualitative achievement of Chapter 9 — turning a timeout into milliseconds — is therefore not unique to structural recovery. A figure of the form “hundreds of times faster than CaDiCaL”, stated without this context, would not survive review.

The advantage is a factor, not a category. On pigeonhole, Hall and exact-*r* the present system is faster by roughly 1.5 to 2.4 times. That is a real difference but a quantitative one between two techniques that both work.

Hidden cardinality separates them. On the hidden pigeonhole family the gap is about an order of magnitude, and the reason is structural rather than incidental: when

the constraint is buried behind auxiliary variables, the graph automorphism computation becomes expensive — BreakID spends over 200 ms before solving begins — whereas resolution-based recovery reaches the constraint directly. This is the one place where the mechanism, not the implementation, differs.

Even so, Section 3 records that recovering hidden cardinality is itself prior art [7]. The honest statement is that the present system is faster than *symmetry breaking* on hidden cardinality; it has not been compared against the semantic detection method that targets the same case.

Capacity-2 goes the other way. BreakID is roughly twice as fast on the capacity-2 family, consistent with Table 9.2, where that family consumes the largest share of the registry budget.

Neither helps on the controls. On the satisfiable control and the random instances both pipelines lose to plain Kissat. Structural machinery is overhead where there is nothing to recover, which is the expected and correct outcome.

10.2 A qualitative difference: what the proof is about

Symmetry breaking modifies the formula. The added constraints preserve satisfiability but not logical equivalence: some satisfying assignments are deliberately excluded. A refutation of the transformed instance is therefore a statement about the transformed instance, and trusting it as a statement about the original requires trusting the transformation.

The refutation of Theorem 4.6 is about the original formula. Every clause used is entailed by Proposition 4.1, and the witness is checked against the original CNF. This is a genuine advantage, although Reeves [13] shows the same ground is being worked for cardinality reasoning generally, and modern symmetry breakers can emit proof certificates of their own.

10.3 Techniques not measured

Two comparisons remain outstanding and are recorded as obligations rather than as results.

Pseudo-Boolean solving. By Theorem 2.7 and Hooker’s correspondence [3], a solver reasoning natively with cardinality constraints refutes these families in polynomial size. RoundingSat [9] is expected to be at least competitive on exact- r and possibly faster. Until measured, no claim of superiority over cutting-plane reasoning is available.

Stronger symmetry breaking. *satsuma* [10] detects a broader class of symmetries than BreakID and runs faster. The comparison in Table 10.1 is therefore against a strong but not current baseline, and BreakID was run with default parameters.

10.4 Machine learning approaches

Learned components inside CDCL improve heuristics by measurable but modest margins; NeuroBack [12] adds a few per cent of solved instances to Kissat. Purely neural solving

has not scaled. Repository-scale LLM code evolution [14] has, and produced solvers beating the human 2025 competition winners.

None of this bears directly on the present work, for the reason given in Section 2.3: these approaches improve an implementation of CDCL, and the obstacle addressed here is a property of the proof system CDCL searches in. An evolved CDCL solver should still be exponential on pigeonhole. The reverse implication is the one to take seriously — implementation-level gains such as those in Chapter 6 are precisely what such systems produce autonomously, so they are the least durable part of this work.

10.5 Summary

Against plain CDCL the system shows large gains on the structural families. Against static symmetry breaking it shows a modest advantage on most of them, a substantial advantage on hidden cardinality, and a disadvantage on capacity-2. Against pseudo-Boolean reasoning no measurement exists and theory favours the competitor. The strongest currently defensible statement is therefore narrow, and Chapter 14 states it in that form.

CHAPTER 11

Phase 6: Demanding Evidence the Proof Does Not Use

11.1 A negative result that pointed at its own cause

The Phase 5 telemetry of Chapter 9 produced a blunt number: over 1,192 public instances, the structural path refuted *none*. Taken alone that number invites the conclusion that recovered cardinality structure is an artefact of generated benchmarks. The stop-point breakdown said something more specific. Recovery never aborted — it completed on all 1,160 instances the gates admitted — so the hidden structure was always reachable. The pipeline died later, and on one family it died in a way that should not have been possible.

The corpus contains SATLIB’s `pigeon-hole` set. The pigeonhole principle is the family this entire technique exists for: Haken’s bound (Theorem 2.6) makes every resolution proof of it exponential, while the counting argument of Theorem 4.6 settles it in one line. On generated pigeonhole instances the engine refuted 14 of 14. On SATLIB’s `hole6.cnf` it reported `no_sound_exact_r_boxes` and found nothing at all.

11.2 The defect

The registry admitted a demand box only when the complete exact- r family was present in the recovered database: both the at-least- r half, consisting of the $\binom{|G|}{|G|-r+1}$ positive clauses, and the at-most- r half, consisting of the $\binom{|G|}{r+1}$ negative ones. The flow network of Section 2.4 contains exactly one edge per box, carrying the demand from the source:

$$c(s, b_i) = r_i.$$

No edge, and therefore no term of the cut, ever encodes $\sum_{v \in B_i} v \leq r_i$. The at-most half was evidence the refutation does not consume, and requiring it is what excluded the archetype. SATLIB encodes PHP_n^{n+1} in the standard way,

$$\underbrace{\bigwedge_{p=1}^{n+1} \left(\bigvee_{h=1}^n x_{p,h} \right)}_{\text{each pigeon occupies some hole}} \quad \wedge \quad \underbrace{\bigwedge_{h=1}^n \bigwedge_{p < q} (\neg x_{p,h} \vee \neg x_{q,h})}_{\text{each hole takes at most one pigeon}} \quad ,$$

and says nothing about a pigeon occupying *at most* one hole, because the refutation does not need it. `hole6.cnf` is precisely 7 positive clauses of width 6 and 126 binary negative clauses. The engine was asking for a constraint the problem has no reason to state.

11.3 One-sided demands are sound

Theorem 11.1 (Refutation from one-sided demands). *Let $B_1, \dots, B_m$ be pairwise disjoint variable sets with $F \models \sum_{v \in B_i} v \geq r_i$, and let $R_1, \dots, R_n$ be pairwise disjoint*

variable sets with $F \models \sum_{v \in R_j} v \leq c_j$, such that $\bigcup_i B_i \subseteq \bigcup_j R_j$. If the bipartite network with source capacities r_i , sink capacities c_j and edge capacities $|B_i \cap R_j|$ admits no flow saturating $\sum_i r_i$, then F is unsatisfiable.

Proof. Suppose $\alpha \models F$ and put $f(i, j) = |\{v \in B_i \cap R_j : \alpha(v) = 1\}|$. Since the R_j are disjoint and cover every box variable, $\sum_j f(i, j) = |\{v \in B_i : \alpha(v) = 1\}| \geq r_i$ by the demand bound. Since the B_i are disjoint, $\sum_i f(i, j) \leq |\{v \in R_j : \alpha(v) = 1\}| \leq c_j$ by the capacity bound. Hence f is a feasible flow saturating every demand, contradicting the hypothesis. Therefore no such α exists. $\square$

The proof reads only lower bounds on the boxes and upper bounds on the resources. An exact- r box satisfies the hypothesis, so Theorem 11.1 strictly generalises Theorem 4.6: the at-most half was never load-bearing. Disjointness and coverage remain indispensable, and both are still enforced — without coverage a box variable outside every resource could meet a demand no capacity constrains.

Phase 6 therefore admits a box proved by its at-least family alone. The witness records which was proved, and `phase2/witness.py` asks for the corresponding evidence: a box marked `atleast` must exhibit the positive family and may not substitute anything for the missing half, while an unmarked box is still held to the full exact- r requirement, so witnesses written before the distinction existed are read unchanged.

11.4 Effect on the public corpus

Table 11.1: PHP_n^{n+1} decided by the Phase 6 front end and by the two locked CDCL baselines. LKK times are front-end time; T/O is the 600 s limit. Instances up to ten holes are SATLIB’s; larger ones are generated in the same encoding.

Pigeons/holes	Source	Vars	Clauses	LKK (ms)	Kissat (s)	CaDiCaL (s)
7/6	SATLIB	42	133	4.83	0.01	0.02
8/7	SATLIB	56	204	6.09	0.03	0.07
9/8	SATLIB	72	297	7.27	0.13	0.54
10/9	SATLIB	90	415	10.61	0.32	3.52
11/10	SATLIB	110	561	13.72	2.52	81.38
12/11	gen.	132	738	14.11	46.00	T/O
13/12	gen.	156	949	24.98	502.42	T/O
14/13	gen.	182	1197	27.76	T/O	T/O

Table 11.1 shows the separation the theory predicts. The front end grows roughly linearly, from 4.83 ms at six holes to 27.76 ms at thirteen. CaDiCaL grows by a factor of about six per hole and needs 81.4 s at ten, where the front end needs 13.7 ms; at eleven holes it exceeds the limit. Kissat survives two holes longer and then also exceeds it. At twelve holes the front end is roughly four orders of magnitude faster than Kissat; at thirteen both baselines fail to answer at all while the front end returns a checked refutation in under 28 ms.

Across the whole corpus the change turns 0 structural refutations into 5, all of them SATLIB pigeonhole instances, each independently validated with total demand = cut capacity + 1. Registry closure rises from 9 to 14 instances and boxes are found on 997 rather than 363. No instance changed between two definite answers.

The cost is real and belongs in the same paragraph as the benefit. Admitting every positive clause as a candidate demand raises the median front-end time from 10.3 ms to 28.6 ms and the corpus total from 627 s to 932 s; the worst single instance pays an extra 27 s. Five refutations on 1,192 instances do not pay for that on a PAR-2 scoreboard. What they establish is narrower and, for the argument of this thesis, more useful: the front end fires on the canonical hard family as distributed by a public benchmark library, not only on instances we generated ourselves.

11.5 What the episode says about the method

Three defects surfaced from a single negative result, and none was found by reading the code.

The first is the one above. The second was in the independent checker: when a witness named an evidence clause the database did not contain, `clauses.index` raised a bare `ValueError` instead of a `WitnessError`, so an invalid witness was reported as a crash rather than as a rejection. It was caught by a Phase 6 test that deliberately strips the `atleast` marker and expects a clean refusal.

The third was methodological and affected the comparison rather than the engine. Both locked baselines refuse SATLIB files outright: the trailing lone 0 that the library appends after the last clause is counted as an extra clause, and `cadical` and `kissat` abort with *too many clauses*. On 408 of 1,192 instances — every `uf` and `uuf` set, `pret`, and one `dubois` — the campaign as configured would have produced no baseline measurement at all. The remedy is a parsing flag on each solver, `-f` and `-relaxed`, which alters DIMACS strictness and not search; Appendix C records the check that the answers are unchanged on instances both solvers already accept.

The common thread is that the measurement, not the reasoning, located every one of them. The *exact-r* requirement had been in the accepted semantics since Phase 2 and had survived every correctness campaign, because the generated benchmarks were written with two-sided constraints and never asked the question.

CHAPTER 12

Phase 5: The Public-Benchmark Campaign

12.1 What was measured

The campaign put five solver configurations against 954 instances of the evaluation split, drawn from SATLIB and the 2024 SAT Competition main track, at a 60s timeout, chosen by the calibration stage from the grid $\{5, 15, 30, 60\}$ under the rule “the smallest timeout within two points of the best solved fraction”: the grid solved 64.7 %, 66.9 %, 69.7 % and 73.1 % respectively, so only the ceiling qualified. Two arms are the locked baselines run standalone; two run the Phase 6 front end with CaDiCaL or Kissat as the fallback; the fifth lets the fitted selector choose between them. In all, 4770 measured runs, followed by a six-variant ablation and the correctness gate.

The corpus split is content-addressed: an instance’s own SHA-256 decides whether it calibrates or evaluates, so nothing tuned on the calibration split can have seen the evaluation split. Timing arms ran sequentially on one core.

12.2 Correctness

Table 12.1: Every instance on which two arms both returned a definite answer, and every instance carrying an externally declared answer.

Check	Result
Instances checked	954
Arms disagreeing with each other	0
Instances with a declared answer	338
Arms disagreeing with the declared answer	0

No pair of arms ever returned two different definite answers, and on every one of the 338 instances whose answer is externally known the campaign agreed with it. The soundness argument of Theorem 11.1 is not contradicted anywhere in 4770 runs, and 3 structural refutations in this split were re-checked by the independent verifier of Chapter 5.

12.3 The headline result

Tables 12.2 and 12.3 say the same thing. Kissat run on its own is the best arm; every configuration carrying the front end is behind it, by between two and six percent. Restricting to SATLIB, where the timeout is not the binding constraint, does not change the ordering in any way that favours the front end.

There is a trap in these numbers worth naming, because the head-to-head figures point the other way. Compared instance by instance against standalone CaDiCaL, the

Table 12.2: PAR-2 over all 954 evaluation instances. Lower is better; the ratio is against the best arm.

Mode	PAR-2 (s)	Ratio	Solved
Kissat	20,746.7	1.000,0	791
LKK + Kissat	21,149.7	1.019,4	787
LKK + selector	21,171.5	1.020,5	787
CaDiCaL	21,630.2	1.042,6	786
LKK + CaDiCaL	22,019.4	1.061,3	781

Table 12.3: The same comparison restricted to SATLIB, excluding the competition instances that time out in every arm.

Mode	PAR-2 (s)	Ratio	Solved
Kissat	1,644.6	1.000,0	755
CaDiCaL	1,708.3	1.038,7	756
LKK + CaDiCaL	1,713.4	1.041,8	755
LKK + selector	1,742.0	1.059,2	754
LKK + Kissat	1,748.0	1.062,9	754

LKK + CaDiCaL arm is faster on 595 instances and slower on 102, with a median advantage of 48.6 ms. It is nevertheless the *worst* arm in Table 12.2. The reason is that it solves 781 instances where standalone CaDiCaL solves 786, and each unsolved instance costs 120s of PAR-2 penalty: five such instances erase 595 wins of 48 ms. Under PAR-2 a small speedup is worth nothing and an extra solved instance is worth everything, so a median head-to-head delta is not evidence of being faster in any sense that matters.

12.4 What the ablation shows

Table 12.4: The front end switched on one stage at a time, over the ablation subset.

Variant	PAR-2 (s)	Ratio	Solved
LKK disabled	5,693.1	1.026,3	75
+ cheap gate	5,699.6	1.027,5	75
+ recovery	5,697.8	1.027,1	75
+ registry	5,694.9	1.026,6	75
+ flow (full)	5,691.3	1.026,0	75
+ selector	5,547.3	1.000,0	76

Table 12.4 is the most direct measurement in this chapter. Going from the front end fully disabled to the complete gate–recovery–registry–flow pipeline moves PAR-2 by -1.8 s out of 5,693 s, a change of 0.03 %, and does not alter the solved count at all. Whatever the structural machinery is doing on this corpus, it is not visible at the resolution of the measurement.

The selector variant scores better, but by one instance: it solves 76 where every other

variant solves 75, and a single escape from a timeout is worth 120s of PAR-2. That is variance, not a component earning its place, and it sits against the selector’s behaviour in the main evaluation, where it chose the better of its two arms 261 times and the worse 410 times. The conclusion the data supports is that the fallback selector should be removed.

12.5 Separating the parser from the structural analysis

Table 12.5: Each arm against standalone CaDiCaL. The first row runs our parser and an in-process CaDiCaL with the front end switched off.

Arm	Instances	Faster	Slower	Median Δ (ms)
front end disabled	75	58	7	−48.8
front end enabled	779	595	102	−48.6

An earlier reading of the head-to-head numbers suggested that LKK + CaDiCaL was beating standalone CaDiCaL, and it was tempting to credit the recovered structure. Table 12.5 rules that out. The `A_lkk_disabled` variant runs the same DIMACS parser and the same in-process CaDiCaL with every structural stage switched off, and it beats the standalone binary by a median of 48.8ms. With the whole front end switched on, the advantage is 48.6ms.

The difference between those two figures is the structural front end’s entire contribution on this corpus. What the comparison actually measures is a parser: reading the file once and handing clauses to CaDiCaL through its API is slightly cheaper than letting the standalone binary read the file itself. That is a real effect and a mundane one, and no claim about cardinality recovery can rest on it.

This is what the control arm was for, and it is the reason a component ablation belongs in any paper that reports a front end’s speedup.

12.6 Where the front end does fire

Against all of the above, the structural path decided 3 instances in this split outright, without invoking a solver at all: the SATLIB pigeonhole family, refuted by the counting argument with a witness the independent checker accepted. Chapter 11 measures that family directly, where the separation runs to four orders of magnitude and both baselines exhaust a 600s budget at thirteen holes.

The campaign therefore fixes the scope rather than refuting the technique. On 954 public instances the front end is neutral — it costs a few milliseconds, saves nothing measurable, and never produces a wrong answer — and on the one family where the theory says a counting argument must beat resolution, it does, by as much as the theory says it should.

CHAPTER 13

Threats to validity and limitations

13.1 The benchmarks are generated

Every measurement in Chapters 9 and 10 is on instances produced by this project’s generators. Two families were included specifically to be unfavourable — the satisfiable capacity control, where structure is present but useless, and random 3-SAT, where there is none — and both behave as an honest evaluation requires. That mitigates but does not remove the concern. A generated corpus reflects the designer’s model of what matters, and a technique tested only on such a corpus has not been shown to transfer.

The corpus of 1 192 public instances described in Table 8.2 was assembled for exactly this reason and the campaign over it has not been run. Until it is, nothing in this report is evidence about real workloads.

13.2 The baseline may be conservative

CaDiCaL 3.0.1 and Kissat 4.0.4 are pinned for reproducibility, but CaDiCaL 2.0 [11] introduced clausal congruence closure, which detects gate structure in the formula and merges equivalent outputs. A current solver therefore performs some structure recovery of its own. The comparison in Table 9.1 should be read as against these specific pinned versions, and a published comparison would need to justify the choice or repeat the measurement against a current release.

13.3 A regression stands

The satisfiable capacity control is slower under Phase 4C than under the sealed engine, consistently across seeds. The cause is understood — honest budget accounting makes the front-end work harder before conceding on an instance where the recovered structure cannot help — but it is a real cost, paid on exactly the class of instance where the technique has nothing to offer. Whether that trade is acceptable depends on the workload mix, which is one of the questions the public campaign is meant to answer.

13.4 The decision procedure is one-sided and incomplete

Theorem 4.6 refutes; it never affirms. A feasible flow means the relaxation is satisfiable, not the formula, so every satisfiable instance is answered by the fallback and pays the front-end cost for nothing.

Incompleteness also enters through the model. The registry requires selected boxes to be pairwise variable-disjoint and every box variable to lie in exactly one selected resource (Section 4.4). Proven constraints that overlap are discarded. Structure that exists and is recovered can therefore still fail to produce a decision, and the reported UNKNOWN outcomes include such cases.

13.5 Budgets bound what can be found

Recovery is limited by depth, clause length, pivot occurrence, derived-clause count, memory and wall time; the registry by the check budget of Chapter 6. These budgets are what make the front-end cheap on instances where it fails, and they are the reason it is not a complete cardinality detector. The budget was fitted on a calibration split (Section 6.3), but a single scalar cannot be right for every family: Table 9.2 shows capacity-2 consuming a large share while random instances consume almost none.

13.6 Cost profile

The front-end costs milliseconds. That suits one-shot decisions on hard instances. It does not suit the regime typical of industrial verification, where a solver is invoked incrementally many thousands of times, each call taking milliseconds and reusing learned clauses. In that setting a millisecond-scale front-end run per call would dominate. No incremental interface has been built or evaluated.

13.7 Statistical strength

The generated campaign uses three seeds and a 5 s timeout on 14 instances. That is adequate to separate the effects reported, which are mostly order-of-magnitude, but it is a small sample. The comparison against BreakID in Table 10.1 is weaker still: single runs, no seed aggregation, default parameters for the competitor. It is reported as indicative and should not be cited as a controlled result.

13.8 Scope of the claims

Results apply to these families, these solver versions, this timeout and this hardware. No complexity-theoretic claim is made or implied: the system exploits a known gap between proof systems (Theorems 2.6 and 2.7) and does not narrow it. Chapter 3 establishes that every component has prior art, and Chapter 10 that the qualitative achievement is shared with symmetry breaking. The contribution claimed is integration, measurement discipline and certification.

CHAPTER 14

Conclusion and outlook

14.1 What was established

A structural front-end recovers exact- r and capacity families from a CNF by bounded resolution, decides the resulting allocation model by max-flow, and emits a witness that an independent checker verifies against the original formula. On the generated corpus it decides 12 of 14 instances faster than both locked baselines, and decides one instance — exact- r at scale 11 — that neither baseline nor the previous engine finishes within the benchmark budget.

The correctness regime is as much a result as the timings. Campaign 20260817Tphase4cfinal04Z passed 5 000 flow cross-checks with 0 mismatches and 46 correctness cases with 0 failures, and every structural refutation was independently re-verified. During development the gate stopped a campaign over a harness defect that would otherwise have entered this report as a false claim (Section 8.2.1).

14.2 The public corpus, and what it took to fire there

The Phase 5 campaign put the front end on 1,192 instances drawn from SATLIB and the SAT Competition. As first configured it refuted none of them. That number was not evidence that the technique is confined to generated benchmarks; it was evidence of a defect, and the telemetry said where. Recovery completed on every admitted instance, so the structure was reachable, yet the registry rejected SATLIB’s pigeonhole set outright — the one family the method exists for.

Chapter 11 gives the cause. The registry required both halves of an exact- r family, while the minimum cut consumes only the lower half; the upper half was evidence the proof never reads, and the standard pigeonhole encoding has no reason to state it. Theorem 11.1 shows that a one-sided demand suffices and strictly generalises Theorem 4.6. With it, public-corpus refutations rise from 0 to 5, every one independently validated, and the front end decides PHP₁₃¹⁴ in under 28 ms where both locked baselines exhaust 600 s.

Two further defects surfaced from the same negative result: the independent checker reported an unprovable witness as a crash rather than a rejection, and both locked baselines refuse 408 of the 1,192 corpus files because of a trailing zero SATLIB appends, which would have left a third of the campaign without any baseline to compare against. None of the three was found by reading the code.

14.3 What the public campaign settled

Chapter 12 put five configurations against 954 public instances, 4770 measured runs, and the answer is unambiguous. Kissat on its own is the best arm; every configuration carrying the front end trails it by two to six percent. Switching the structural pipeline on one stage at a time moves PAR-2 by 0.03 % and changes the solved count not at all. A control arm that runs our parser and an in-process CaDiCaL with the front end *disabled* beats the standalone binary by a median of 48.8 ms, against 48.6 ms with the front end

enabled — so the advantage that first looked like a structural result is a property of reading the file once and passing clauses through an API. The fallback selector chose the worse of its two arms more often than the better and should be removed.

What did not move is correctness: 0 disagreements between arms across 4770 runs, and 0 disagreements with the 338 instances whose answer is externally known.

So the technique is neutral on general public benchmarks — it costs a few milliseconds, returns nothing measurable, and is never wrong — while on the pigeonhole family it decides instances outright that neither baseline finishes. That is a scope, established by measurement rather than asserted, and it is the honest shape of the result.

14.4 The defensible claim

Stated carefully, and with the qualifications of Chapters 10 and 13:

Cardinality structure recovered from a CNF can be *decided* rather than re-encoded. LKK-HSAT assembles recovered exact- r and capacity families into an allocation model and refutes it by max-flow, answering — with a certificate about the original formula — instances on which conflict-driven clause learning is provably exponential and on which the locked baselines exhaust the benchmark budget. The recovery step has established antecedents; the decision procedure it feeds and the certificate it admits do not. The advantage is confined to instances whose obstruction is a counting argument: measured over 954 public benchmarks the front end neither gains nor loses, and the claim is one of scope, not of general speedup.

This answers a question the closest prior work left open. Biere et al. [7] solved detection and wrote that using the recovered information to improve solving time was future work; the standard answers since have been re-encoding or delegation to a pseudo-Boolean solver, both of which keep a search in the loop. Deciding the structure directly removes it.

Three statements are *not* supported and should not be made. That the system is hundreds of times faster than modern SAT solving in general: it is faster than the locked baselines on these families, but Chapter 10 shows symmetry breaking reaching the same instances by another route. That it is the first system to recover hidden cardinality: Biere et al. did so in 2014. That the advantage transfers to real workloads: that campaign is prepared and unrun.

14.5 Immediate obligations

Three measurements would materially change what can be claimed.

1. **The public campaign.** 1 192 instances are acquired, hash-verified and prepared. Running it is the difference between a result on generated benchmarks and a result on standard ones, and it may well be negative: the front-end's cost is paid on every instance, and most public instances carry no recoverable cardinality structure.
2. **Pseudo-Boolean comparison.** RoundingSat [9] reasons in the system that makes these families easy. If it dominates on exact- r , the honest position becomes that this work reaches cutting-plane capability through CNF without matching a native cutting-plane solver.

3. **A current baseline.** Repeating the comparison against a CaDiCaL release with congruence closure [11], and against satsuma [10] rather than BreakID.

14.6 Technical directions

Table 9.2 localises the remaining cost in the registry, and specifically in clique growth on the capacity-2 family, which consumes the largest share of the budget. Section 4.4’s disjointness requirement discards proven constraints and is a candidate for relaxation, at the price of a more complex flow model. A single scalar budget is also demonstrably crude: the same value that is generous for random instances binds tightly on capacity-2, which suggests a structure-dependent budget fitted the way Section 6.3 fits the current one.

A more speculative direction follows from Section 4.5. The current procedure decides the whole allocation model or nothing. A partial model, obtained when the registry cannot cover every variable, still supports conditional reasoning: assuming a literal and re-running the flow can refute that literal, which is a clause the formula entails. Preliminary work in this direction found that on the present families such conditional refutations are either degenerate or absent, but the mechanism is sound and may apply elsewhere.

14.7 A note on durability

Chapter 6 describes implementation improvements that produced large measured gains. Section 10 observes that LLM-driven code evolution now produces comparable gains autonomously and has beaten human competition winners. The implication is not that such work is worthless but that it is not where the durable contribution lies. What resists that pressure is the part that changes the reasoning rather than the implementation: recovering a constraint that the encoding destroyed, deciding it in a system where it is tractable, and being able to prove it afterwards.

APPENDIX A

Full correctness data

Table A.1 lists every case of the correctness campaign of run 20260817Tphase4cfinal04Z. Columns give the expected answer, the structural verdict of the sealed Phase 4A engine, the structural verdict of the Phase 4C engine, the verdict of the independent Python reference implementation, the answers of both locked solvers, and whether the case passed every check of Section 8.2.

A case passes only when the definite answers agree across all four implementations, the Phase 4C engine does not contradict the sealed engine, no stage before the registry changed, and any emitted witness verified.

Table A.1: Correctness campaign, all cases.

Instance	Expected	4A	4C	Python	CaDiCaL	Kissat	Pass
001 pigeonhole exact1	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	yes
002 pigeonhole exact1	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	yes
003 pigeonhole exact1 ⁺	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	yes
004 pigeonhole exact1 ⁺	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	yes
005 exact r allocation	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	yes
006 exact r allocation	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	yes
007 exact r allocation ⁺	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	yes
008 exact r allocation ⁺	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	yes
009 bipartite matching	SAT	FEAS	FEAS	FEAS	SAT	SAT	yes
010 bipartite matching	SAT	FEAS	FEAS	FEAS	SAT	SAT	yes
011 bipartite matching ⁺	SAT	FEAS	FEAS	FEAS	SAT	SAT	yes
012 bipartite matching ⁺	SAT	FEAS	FEAS	FEAS	SAT	SAT	yes
013 overlapping hall violation	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	yes
014 overlapping hall violation	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	yes
015 overlapping hall violation ⁺	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	yes
016 overlapping hall violation ⁺	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	yes
017 satisfiable capacity contr	SAT	FEAS	FEAS	FEAS	SAT	SAT	yes
018 satisfiable capacity contr	SAT	FEAS	FEAS	FEAS	SAT	SAT	yes
019 satisfiable capacity contr ⁺	SAT	FEAS	FEAS	FEAS	SAT	SAT	yes
020 satisfiable capacity contr ⁺	SAT	FEAS	FEAS	FEAS	SAT	SAT	yes
021 capacity violation	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	yes
022 capacity violation	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	yes
023 capacity violation ⁺	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	yes
024 capacity violation ⁺	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	yes
025 hidden cardinality	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	yes
026 hidden cardinality	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	yes
027 hidden cardinality ⁺	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	yes
028 hidden cardinality ⁺	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	yes
029 recovery budget abort cont	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	yes
030 recovery budget abort cont	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	yes
031 recovery budget abort cont ⁺	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	yes
032 recovery budget abort cont ⁺	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	yes
001 pigeonhole exact1 s8	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	yes

continued on the next page

Table A.1 continued

Instance	Expected	4A	4C	Python	CaDiCaL	Kissat	Pass
002 pigeonhole exact1 s10	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	yes
003 exact r allocation s9	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	yes
004 exact r allocation s11	UNSAT	unk	UNSAT	UNSAT	UNSAT	UNSAT	yes
005 overlapping hall violation s8	UNSAT	unk	UNSAT	UNSAT	UNSAT	UNSAT	yes
006 overlapping hall violation s9	UNSAT	unk	UNSAT	UNSAT	UNSAT	UNSAT	yes
007 capacity2 violation s5	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	yes
008 capacity2 violation s5 ⁺	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	yes
009 hidden pigeonhole s9	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	yes
010 hidden pigeonhole s9 ⁺	UNSAT	unk	UNSAT	UNSAT	UNSAT	UNSAT	yes
011 satisfiable capacity contr s24	SAT	unk	unk	unk	SAT	SAT	yes
012 random 3sat s150	SAT	unk	unk	unk	SAT	SAT	yes
013 random 3sat s400	SAT	unk	unk	unk	SAT	SAT	yes
014 random 3sat s800	SAT	unk	unk	unk	SAT	SAT	yes

Reading the structural columns

A dash means the engine produced no structural verdict: either a gate declined, recovery aborted, the registry reported UNKNOWN, or — in the case of the sealed engine on the hardest instance — the run did not finish within the validation budget. In every such case the instance was answered by the fallback solver on the unchanged formula, so a dash indicates the absence of a structural decision, never an incorrect one.

The Python column is an independent reimplementaion of the registry search used solely as a cross-check. It shares no code with the C++ engine; the two are compared on both the verdict and the number of checks consumed, and the harness incident of Section 8.2.1 was detected precisely by this comparison.

APPENDIX B

Complete measurement data

Table B.1 lists every individual solver invocation of campaign 20260817Tphase4cfinal04Z: 14 instances, three seeds, four modes. The aggregates of Chapter 9 are means over the three seeds of the PAR-2 column, and the structural column records the verdict of the front-end where one was reached.

Rows with an empty structural column are the locked baselines, which have no front-end. A dash for an LKK mode means the front-end reached no structural verdict and the instance was answered by the fallback on the unchanged formula.

Table B.1: Every solver invocation of the campaign.

Instance	Seed	Solver	Status	Runtime	PAR-2	Structural
001 pigeonhole s8	1	CaDiCaL	UNSAT	119.0	119.0	—
001 pigeonhole s8	1	Kissat	UNSAT	67.7	67.7	—
001 pigeonhole s8	1	LKK 4A	UNSAT	35.5	35.5	UNSAT
001 pigeonhole s8	1	LKK 4C	UNSAT	21.0	21.0	UNSAT
001 pigeonhole s8	2	CaDiCaL	UNSAT	119.8	119.8	—
001 pigeonhole s8	2	Kissat	UNSAT	68.3	68.3	—
001 pigeonhole s8	2	LKK 4A	UNSAT	38.6	38.6	UNSAT
001 pigeonhole s8	2	LKK 4C	UNSAT	19.8	19.8	UNSAT
001 pigeonhole s8	3	CaDiCaL	UNSAT	119.9	119.9	—
001 pigeonhole s8	3	Kissat	UNSAT	70.0	70.0	—
001 pigeonhole s8	3	LKK 4A	UNSAT	36.5	36.5	UNSAT
001 pigeonhole s8	3	LKK 4C	UNSAT	20.7	20.7	UNSAT
002 pigeonhole s10	1	CaDiCaL	UNSAT	4,485.7	4,485.7	—
002 pigeonhole s10	1	Kissat	UNSAT	418.8	418.8	—
002 pigeonhole s10	1	LKK 4A	UNSAT	69.5	69.5	UNSAT
002 pigeonhole s10	1	LKK 4C	UNSAT	18.5	18.5	UNSAT
002 pigeonhole s10	2	CaDiCaL	UNSAT	4,384.1	4,384.1	—
002 pigeonhole s10	2	Kissat	UNSAT	569.1	569.1	—
002 pigeonhole s10	2	LKK 4A	UNSAT	119.0	119.0	UNSAT
002 pigeonhole s10	2	LKK 4C	UNSAT	18.2	18.2	UNSAT
002 pigeonhole s10	3	CaDiCaL	UNSAT	4,988.5	4,988.5	—
002 pigeonhole s10	3	Kissat	UNSAT	519.8	519.8	—
002 pigeonhole s10	3	LKK 4A	UNSAT	117.3	117.3	UNSAT
002 pigeonhole s10	3	LKK 4C	UNSAT	19.0	19.0	UNSAT
003 exact-r s9	1	CaDiCaL	UNSAT	218.0	218.0	—
003 exact-r s9	1	Kissat	UNSAT	168.0	168.0	—
003 exact-r s9	1	LKK 4A	UNSAT	68.3	68.3	UNSAT
003 exact-r s9	1	LKK 4C	UNSAT	19.2	19.2	UNSAT
003 exact-r s9	2	CaDiCaL	UNSAT	217.8	217.8	—
003 exact-r s9	2	Kissat	UNSAT	218.9	218.9	—
003 exact-r s9	2	LKK 4A	UNSAT	34.7	34.7	UNSAT
003 exact-r s9	2	LKK 4C	UNSAT	19.4	19.4	UNSAT
003 exact-r s9	3	CaDiCaL	UNSAT	217.7	217.7	—
003 exact-r s9	3	Kissat	UNSAT	167.9	167.9	—

continued on the next page

Table B.1 continued

Instance	Seed	Solver	Status	Runtime	PAR-2	Structural
003 exact- <i>r</i> s9	3	LKK 4A	UNSAT	35.4	35.4	UNSAT
003 exact- <i>r</i> s9	3	LKK 4C	UNSAT	19.1	19.1	UNSAT
004 exact- <i>r</i> s11	1	CaDiCaL	t/o	5,038.9	10,000.0	—
004 exact- <i>r</i> s11	1	Kissat	t/o	5,038.7	10,000.0	—
004 exact- <i>r</i> s11	1	LKK 4A	t/o	5,037.8	10,000.0	—
004 exact- <i>r</i> s11	1	LKK 4C	UNSAT	18.6	18.6	UNSAT
004 exact- <i>r</i> s11	2	CaDiCaL	t/o	5,037.7	10,000.0	—
004 exact- <i>r</i> s11	2	Kissat	t/o	5,035.4	10,000.0	—
004 exact- <i>r</i> s11	2	LKK 4A	t/o	5,037.2	10,000.0	—
004 exact- <i>r</i> s11	2	LKK 4C	UNSAT	18.1	18.1	UNSAT
004 exact- <i>r</i> s11	3	CaDiCaL	t/o	5,036.7	10,000.0	—
004 exact- <i>r</i> s11	3	Kissat	t/o	5,036.9	10,000.0	—
004 exact- <i>r</i> s11	3	LKK 4A	t/o	5,036.4	10,000.0	—
004 exact- <i>r</i> s11	3	LKK 4C	UNSAT	17.9	17.9	UNSAT
005 Hall s8	1	CaDiCaL	UNSAT	569.9	569.9	—
005 Hall s8	1	Kissat	UNSAT	117.7	117.7	—
005 Hall s8	1	LKK 4A	UNSAT	821.8	821.8	UNKNOWN
005 Hall s8	1	LKK 4C	UNSAT	19.0	19.0	UNSAT
005 Hall s8	2	CaDiCaL	UNSAT	520.3	520.3	—
005 Hall s8	2	Kissat	UNSAT	220.5	220.5	—
005 Hall s8	2	LKK 4A	UNSAT	771.1	771.1	UNKNOWN
005 Hall s8	2	LKK 4C	UNSAT	35.9	35.9	UNSAT
005 Hall s8	3	CaDiCaL	UNSAT	470.2	470.2	—
005 Hall s8	3	Kissat	UNSAT	117.6	117.6	—
005 Hall s8	3	LKK 4A	UNSAT	770.8	770.8	UNKNOWN
005 Hall s8	3	LKK 4C	UNSAT	36.8	36.8	UNSAT
006 Hall s9	1	CaDiCaL	UNSAT	4,334.5	4,334.5	—
006 Hall s9	1	Kissat	UNSAT	368.5	368.5	—
006 Hall s9	1	LKK 4A	UNSAT	4,335.5	4,335.5	UNKNOWN
006 Hall s9	1	LKK 4C	UNSAT	35.2	35.2	UNSAT
006 Hall s9	2	CaDiCaL	UNSAT	4,788.7	4,788.7	—
006 Hall s9	2	Kissat	UNSAT	368.9	368.9	—
006 Hall s9	2	LKK 4A	UNSAT	4,435.9	4,435.9	UNKNOWN
006 Hall s9	2	LKK 4C	UNSAT	37.3	37.3	UNSAT
006 Hall s9	3	CaDiCaL	UNSAT	3,280.5	3,280.5	—
006 Hall s9	3	Kissat	UNSAT	369.1	369.1	—
006 Hall s9	3	LKK 4A	UNSAT	3,582.6	3,582.6	UNKNOWN
006 Hall s9	3	LKK 4C	UNSAT	35.3	35.3	UNSAT
007 capacity-2 s5	1	CaDiCaL	UNSAT	670.1	670.1	—
007 capacity-2 s5	1	Kissat	UNSAT	268.6	268.6	—
007 capacity-2 s5	1	LKK 4A	UNSAT	268.6	268.6	UNSAT
007 capacity-2 s5	1	LKK 4C	UNSAT	67.8	67.8	UNSAT
007 capacity-2 s5	2	CaDiCaL	UNSAT	1,122.0	1,122.0	—
007 capacity-2 s5	2	Kissat	UNSAT	319.2	319.2	—
007 capacity-2 s5	2	LKK 4A	UNSAT	269.6	269.6	UNSAT
007 capacity-2 s5	2	LKK 4C	UNSAT	118.5	118.5	UNSAT
007 capacity-2 s5	3	CaDiCaL	UNSAT	1,776.6	1,776.6	—
007 capacity-2 s5	3	Kissat	UNSAT	320.0	320.0	—
007 capacity-2 s5	3	LKK 4A	UNSAT	269.1	269.1	UNSAT

continued on the next page

Table B.1 continued

Instance	Seed	Solver	Status	Runtime	PAR-2	Structural
007 capacity-2 $s5$	3	LKK 4C	UNSAT	67.0	67.0	UNSAT
008 capacity-2 $s5^+$	1	CaDiCaL	UNSAT	1,974.9	1,974.9	—
008 capacity-2 $s5^+$	1	Kissat	UNSAT	219.0	219.0	—
008 capacity-2 $s5^+$	1	LKK 4A	UNSAT	420.1	420.1	UNSAT
008 capacity-2 $s5^+$	1	LKK 4C	UNSAT	68.2	68.2	UNSAT
008 capacity-2 $s5^+$	2	CaDiCaL	UNSAT	1,576.0	1,576.0	—
008 capacity-2 $s5^+$	2	Kissat	UNSAT	268.2	268.2	—
008 capacity-2 $s5^+$	2	LKK 4A	UNSAT	419.8	419.8	UNSAT
008 capacity-2 $s5^+$	2	LKK 4C	UNSAT	118.2	118.2	UNSAT
008 capacity-2 $s5^+$	3	CaDiCaL	UNSAT	1,223.0	1,223.0	—
008 capacity-2 $s5^+$	3	Kissat	UNSAT	318.9	318.9	—
008 capacity-2 $s5^+$	3	LKK 4A	UNSAT	368.9	368.9	UNSAT
008 capacity-2 $s5^+$	3	LKK 4C	UNSAT	117.6	117.6	UNSAT
009 hidden pigeonhole $s9$	1	CaDiCaL	UNSAT	418.6	418.6	—
009 hidden pigeonhole $s9$	1	Kissat	UNSAT	117.8	117.8	—
009 hidden pigeonhole $s9$	1	LKK 4A	UNSAT	67.3	67.3	UNSAT
009 hidden pigeonhole $s9$	1	LKK 4C	UNSAT	18.4	18.4	UNSAT
009 hidden pigeonhole $s9$	2	CaDiCaL	UNSAT	569.6	569.6	—
009 hidden pigeonhole $s9$	2	Kissat	UNSAT	168.4	168.4	—
009 hidden pigeonhole $s9$	2	LKK 4A	UNSAT	67.7	67.7	UNSAT
009 hidden pigeonhole $s9$	2	LKK 4C	UNSAT	35.4	35.4	UNSAT
009 hidden pigeonhole $s9$	3	CaDiCaL	UNSAT	470.2	470.2	—
009 hidden pigeonhole $s9$	3	Kissat	UNSAT	117.9	117.9	—
009 hidden pigeonhole $s9$	3	LKK 4A	UNSAT	68.3	68.3	UNSAT
009 hidden pigeonhole $s9$	3	LKK 4C	UNSAT	35.1	35.1	UNSAT
010 hidden pigeonhole $s9^+$	1	CaDiCaL	UNSAT	569.6	569.6	—
010 hidden pigeonhole $s9^+$	1	Kissat	UNSAT	169.1	169.1	—
010 hidden pigeonhole $s9^+$	1	LKK 4A	UNSAT	519.9	519.9	UNKNOWN
010 hidden pigeonhole $s9^+$	1	LKK 4C	UNSAT	35.7	35.7	UNSAT
010 hidden pigeonhole $s9^+$	2	CaDiCaL	UNSAT	620.6	620.6	—
010 hidden pigeonhole $s9^+$	2	Kissat	UNSAT	169.9	169.9	—
010 hidden pigeonhole $s9^+$	2	LKK 4A	UNSAT	570.5	570.5	UNKNOWN
010 hidden pigeonhole $s9^+$	2	LKK 4C	UNSAT	37.2	37.2	UNSAT
010 hidden pigeonhole $s9^+$	3	CaDiCaL	UNSAT	670.9	670.9	—
010 hidden pigeonhole $s9^+$	3	Kissat	UNSAT	170.2	170.2	—
010 hidden pigeonhole $s9^+$	3	LKK 4A	UNSAT	571.3	571.3	UNKNOWN
010 hidden pigeonhole $s9^+$	3	LKK 4C	UNSAT	36.4	36.4	UNSAT
011 SAT control $s24$	1	CaDiCaL	SAT	118.3	118.3	—
011 SAT control $s24$	1	Kissat	SAT	36.1	36.1	—

continued on the next page

Table B.1 continued

Instance	Seed	Solver	Status	Runtime	PAR-2	Structural
011 SAT control <i>s</i> 24	1	LKK 4A	SAT	68.4	68.4	UNKNOWN
011 SAT control <i>s</i> 24	1	LKK 4C	SAT	118.7	118.7	UNKNOWN
011 SAT control <i>s</i> 24	2	CaDiCaL	SAT	117.9	117.9	—
011 SAT control <i>s</i> 24	2	Kissat	SAT	35.9	35.9	—
011 SAT control <i>s</i> 24	2	LKK 4A	SAT	118.5	118.5	UNKNOWN
011 SAT control <i>s</i> 24	2	LKK 4C	SAT	118.4	118.4	UNKNOWN
011 SAT control <i>s</i> 24	3	CaDiCaL	SAT	118.1	118.1	—
011 SAT control <i>s</i> 24	3	Kissat	SAT	35.5	35.5	—
011 SAT control <i>s</i> 24	3	LKK 4A	SAT	68.6	68.6	UNKNOWN
011 SAT control <i>s</i> 24	3	LKK 4C	SAT	120.0	120.0	UNKNOWN
012 random 3-SAT <i>s</i> 150	1	CaDiCaL	SAT	117.8	117.8	—
012 random 3-SAT <i>s</i> 150	1	Kissat	SAT	117.6	117.6	—
012 random 3-SAT <i>s</i> 150	1	LKK 4A	SAT	68.1	68.1	UNKNOWN
012 random 3-SAT <i>s</i> 150	1	LKK 4C	SAT	68.7	68.7	UNKNOWN
012 random 3-SAT <i>s</i> 150	2	CaDiCaL	SAT	118.4	118.4	—
012 random 3-SAT <i>s</i> 150	2	Kissat	SAT	117.9	117.9	—
012 random 3-SAT <i>s</i> 150	2	LKK 4A	SAT	67.7	67.7	UNKNOWN
012 random 3-SAT <i>s</i> 150	2	LKK 4C	SAT	69.1	69.1	UNKNOWN
012 random 3-SAT <i>s</i> 150	3	CaDiCaL	SAT	119.3	119.3	—
012 random 3-SAT <i>s</i> 150	3	Kissat	SAT	66.8	66.8	—
012 random 3-SAT <i>s</i> 150	3	LKK 4A	SAT	68.4	68.4	UNKNOWN
012 random 3-SAT <i>s</i> 150	3	LKK 4C	SAT	67.8	67.8	UNKNOWN
013 random 3-SAT <i>s</i> 400	1	CaDiCaL	SAT	821.9	821.9	—
013 random 3-SAT <i>s</i> 400	1	Kissat	SAT	168.9	168.9	—
013 random 3-SAT <i>s</i> 400	1	LKK 4A	SAT	871.5	871.5	UNKNOWN
013 random 3-SAT <i>s</i> 400	1	LKK 4C	SAT	821.1	821.1	UNKNOWN
013 random 3-SAT <i>s</i> 400	2	CaDiCaL	SAT	871.3	871.3	—
013 random 3-SAT <i>s</i> 400	2	Kissat	SAT	2,075.8	2,075.8	—
013 random 3-SAT <i>s</i> 400	2	LKK 4A	SAT	770.6	770.6	UNKNOWN
013 random 3-SAT <i>s</i> 400	2	LKK 4C	SAT	821.8	821.8	UNKNOWN
013 random 3-SAT <i>s</i> 400	3	CaDiCaL	SAT	921.8	921.8	—
013 random 3-SAT <i>s</i> 400	3	Kissat	SAT	772.1	772.1	—
013 random 3-SAT <i>s</i> 400	3	LKK 4A	SAT	820.8	820.8	UNKNOWN
013 random 3-SAT <i>s</i> 400	3	LKK 4C	SAT	872.6	872.6	UNKNOWN
014 random 3-SAT <i>s</i> 800	1	CaDiCaL	SAT	719.9	719.9	—
014 random 3-SAT <i>s</i> 800	1	Kissat	SAT	117.9	117.9	—
014 random 3-SAT <i>s</i> 800	1	LKK 4A	SAT	973.2	973.2	UNKNOWN
014 random 3-SAT <i>s</i> 800	1	LKK 4C	SAT	822.1	822.1	UNKNOWN
014 random 3-SAT <i>s</i> 800	2	CaDiCaL	SAT	719.3	719.3	—
014 random 3-SAT <i>s</i> 800	2	Kissat	SAT	168.0	168.0	—
014 random 3-SAT <i>s</i> 800	2	LKK 4A	SAT	770.3	770.3	UNKNOWN
014 random 3-SAT <i>s</i> 800	2	LKK 4C	SAT	771.2	771.2	UNKNOWN
014 random 3-SAT <i>s</i> 800	3	CaDiCaL	SAT	1,221.9	1,221.9	—
014 random 3-SAT <i>s</i> 800	3	Kissat	SAT	168.4	168.4	—
014 random 3-SAT <i>s</i> 800	3	LKK 4A	SAT	1,172.4	1,172.4	UNKNOWN
014 random 3-SAT <i>s</i> 800	3	LKK 4C	SAT	1,122.0	1,122.0	UNKNOWN

APPENDIX C

Reproduction

C.1 Environment

Every solver runs inside a container built from a digest-pinned Debian base. CaDiCaL 3.0.1 (commit c607304) and Kissat 4.0.4 (commit 8af8e56) are compiled from source with `-O3 -DNDEBUG`; the structural engine is compiled with `g++ -std=c++17 -O3 -DNDEBUG -Wall -Wextra`. Each run records the SHA-256 of every binary it used, the image identifier, and the tree digests of all predecessor phases, in its own `metadata.json`.

C.2 Commands

Table C.1: Reproduction commands. Each writes a timestamped, immutable run directory; existing runs are never overwritten.

Step	Command
Build the engine	<code>docker build -f Dockerfile.phase4c -t lkk-hsat-phase4c:native .</code>
Core drift test	<code>python3 -m unittest native.core.test_core_drift</code>
Accounting invariants	<code>python3 -m unittest phase4c.test_accounting</code>
Phase 4C campaign	<code>powershell -File .\scripts\run_phase4c.ps1</code>
Budget tuning	<code>python3 -m phase5.tune -output results/tuning</code>
Rival comparison	<code>docker build -f Dockerfile.rivals -t lkk-hsat-rivals:native .</code>
Public corpus	<code>python3 -m phase5.acquire</code>
Phase 5 campaign	<code>powershell -File .\scripts\run_phase5.ps1</code>
Build this report	<code>powershell -File .\scripts\build_paper.ps1</code>

The full regression suite spans every phase:

```
python3 -m unittest benchmark.test_harness phase2.test_phase2
phase3.test_phase3 phase4a.test_phase4a
phase4b.test_phase4b phase4c.test_phase4c
phase4c.test_accounting phase5.test_phase5
native.core.test_core_drift
```

C.3 Artifacts of a campaign

A completed run directory contains `RESULTS.md`; `summary.csv` with per-instance aggregates; `measurements.csv` with every individual run; `correctness.csv` with the gate results; `metadata.json` with environment, digests and counts; and `logs/` with the raw output of every solver invocation, alongside `failures/` preserving any case that did not pass.

C.4 How this document is built

The prose of this report is written by hand; every number in it is not. The generator `report/build_paper.py` reads the campaign CSVs and emits the table bodies and a set of $\text{\LaTeX}$ macros for the single-value figures quoted in the text. Rebuilding after a new campaign changes the tables and every derived figure together, so the document cannot drift away from the data it describes. The reference campaign for this build is `20260817Tphase4cfinal04Z`.

C.5 Data availability

All campaign artifacts, the generators for the benchmark families, the witness format and its checker, and the acquisition manifests for the public corpus are contained in the repository. The public corpus itself is not redistributed; it is fetched from SATLIB and from the SAT Competition 2024 main track archive (Zenodo, [10.5281/zenodo.15095752](https://doi.org/10.5281/zenodo.15095752), CC-BY-4.0) and verified against published checksums, with every instance’s own SHA-256 recorded in the manifest.

APPENDIX D

A worked example

This appendix traces the pipeline on a small instance, so that every definition of Chapter 4 can be seen acting on concrete data. The instance is PHP_3^4 : four pigeons, three holes.

D.1 The formula

Variables are $x_{i,j}$ for $i \in \{1, \dots, 4\}$, $j \in \{1, 2, 3\}$, numbered $v = 3(i - 1) + j$, so pigeon 1 owns $\{1, 2, 3\}$, pigeon 2 owns $\{4, 5, 6\}$, and so on. Following Equation (2.4), the formula is

$$\underbrace{\bigwedge_{i=1}^4 (x_{i,1} \vee x_{i,2} \vee x_{i,3})}_{4 \text{ clauses}} \wedge \underbrace{\bigwedge_{j=1}^3 \bigwedge_{1 \leq i < i' \leq 4} (\neg x_{i,j} \vee \neg x_{i',j})}_{3 \cdot \binom{4}{2} = 18 \text{ clauses}},$$

a total of 12 variables and 22 clauses.

D.2 Stage 1: the cheap gate

Scanning once gives $p = 4$ monotone positive clauses (the pigeon rows), $n = 18$ monotone negative clauses (the hole pairs) and $\mu = 0$ mirror pairs, since no clause appears in both polarities over a shared remainder. Condition (4.1) is satisfied through $p > 0 \wedge n > 0$: both directions of a cardinality constraint are represented. The gate admits the formula.

D.3 Stage 3: recovery

Every pigeon row already has the form of an AtLeast_1 clause and every hole pair the form of an AtMost_1 clause, so nothing is hidden here and recovery adds little: with the pivot-occurrence limit in force, no variable qualifies as an encoding auxiliary. We therefore have $\mathcal{D} = \mathcal{C}(F)$. The hidden variant of this family, discussed in Section 2.5, is where recovery does the work; it is omitted here to keep the trace short.

D.4 Stage 4: the registry

Box candidates. A candidate arises from a monotone positive clause p and a monotone negative clause n sharing exactly two variables. Take $p = (x_{1,1} \vee x_{1,2} \vee x_{1,3}) = \{1, 2, 3\}$ and $n = (\neg x_{1,1} \vee \neg x_{1,2}) = \{1, 2\}$. Their variable sets share $\{1, 2\}$, exactly two, so

$$G = \text{var}(p) \cup \text{var}(n) = \{1, 2, 3\}, \quad r = |n| - 1 = 1.$$

The acceptance test of Definition 4.2 requires $\mathcal{F}_{=1}(\{1, 2, 3\}) \subseteq \mathcal{D}$, that is

$$\underbrace{\{-1 \vee \neg 2, \neg 1 \vee \neg 3, \neg 2 \vee \neg 3\}}_{\text{AtMost}_1, \binom{3}{2}=3} \cup \underbrace{\{1 \vee 2 \vee 3\}}_{\text{AtLeast}_1, \binom{3}{3}=1},$$

four clauses by Equation (2.3). All four are present — the three hole-pair clauses restricted to pigeon 1 and the pigeon-1 row — so the candidate is accepted. The same holds for each of the four pigeons, giving boxes

$$(\{1, 2, 3\}, 1), \quad (\{4, 5, 6\}, 1), \quad (\{7, 8, 9\}, 1), \quad (\{10, 11, 12\}, 1).$$

They are pairwise variable-disjoint, so all four are selected.

Resource candidates. With $s = 2$, the edges are the monotone negative clauses of size two whose variables lie in distinct boxes. For hole 1 these are the pairs drawn from $\{1, 4, 7, 10\}$, and all $\binom{4}{2} = 6$ are present, so that set forms a clique and yields the resource $(\{1, 4, 7, 10\}, 1)$. Holes 2 and 3 give $(\{2, 5, 8, 11\}, 1)$ and $(\{3, 6, 9, 12\}, 1)$.

Every box variable now lies in exactly one resource, so the registry reports success.

D.5 Stage 5: the flow test

Definition 4.5 builds the network with four box nodes of demand 1, three resource nodes of capacity 1, and an arc from each box to each resource, since every pigeon can occupy every hole. Total demand is

$$\Delta = \sum_{i=1}^4 r_i = 4.$$

The three sink arcs carry at most $1 + 1 + 1 = 3$ units in total, so the maximum flow is $3 < 4 = \Delta$. By Theorem 4.6 the formula is unsatisfiable.

The minimum cut consists of the three resource-to-sink arcs, of capacity 3, matching the maximum flow as Theorem 2.9 requires. Reading it through Corollary 4.7: the four boxes on the source side have combined demand 4, while the resources reachable from them supply only 3 — Hall’s condition (Theorem 2.10) fails on the set of all four pigeons. This is the pigeonhole argument, recovered from clauses and decided by counting.

D.6 The certificate

The witness records the 22 database clauses, all marked original; the four boxes with the identifiers of their four evidence clauses each; the three resources with six evidence clauses each; the variable-to-resource map; and the flow figures $\Delta = 4$, maximum flow 3, with the cut listing all four boxes and all three resources on the source side and a cut capacity of 3.

Checking it, per Chapter 5: pass 1 confirms all 22 clauses lie in F ; pass 2 regenerates $\mathcal{F}_{=1}(G)$ for each box and $\text{AtMost}_1(H)$ for each resource and finds all present; pass 3 confirms disjointness and the injectivity of the variable-to-resource map on each box; pass 4 rebuilds the network and confirms the cut has capacity $3 < 4$. The refutation stands by Proposition 5.1.

D.7 Contrast: where the procedure declines

Change the instance to three pigeons and three holes. The construction proceeds identically until the flow test, where $\Delta = 3$ and the maximum flow is also 3: the model is feasible. The engine emits no witness and hands the unchanged formula to the fallback solver, which reports satisfiable.

This is the asymmetry noted in Section 4.5. A feasible flow means the recovered relaxation admits an assignment; it does not mean the formula does, since the relaxation captures only the recovered constraints. The procedure can refute and never affirm, and the cost of the front-end on this instance is paid for nothing — the situation the satisfiable capacity control of Chapter 8 is designed to measure.

Bibliography

- [1] A. Haken. The intractability of resolution. *Theoretical Computer Science*, 39:297–308, 1985.
- [2] W. Cook, C. R. Coullard, and G. Turán. On the complexity of cutting-plane proofs. *Discrete Applied Mathematics*, 18(1):25–38, 1987.
- [3] J. N. Hooker. Generalized resolution and cutting planes. *Annals of Operations Research*, 12(1–4):217–239, 1988.
- [4] P. Hall. On representatives of subsets. *Journal of the London Mathematical Society*, 10(1):26–30, 1935.
- [5] J.-C. Régin. A filtering algorithm for constraints of difference in CSPs. In *Proceedings of the Twelfth National Conference on Artificial Intelligence (AAAI)*, pages 362–367, 1994.
- [6] R. Ostrowski, É. Grégoire, B. Mazure, and L. Saïs. Recovering and exploiting structural knowledge from CNF formulas. In *Principles and Practice of Constraint Programming (CP)*, volume 2470 of *LNCS*, pages 185–199. Springer, 2002.
- [7] A. Biere, D. Le Berre, E. Lonca, and N. Manthey. Detecting cardinality constraints in CNF. In *Theory and Applications of Satisfiability Testing (SAT)*, volume 8561 of *LNCS*, pages 285–301. Springer, 2014.
- [8] J. Devriendt, B. Bogaerts, M. Bruynooghe, and M. Denecker. Improved static symmetry breaking for SAT. In *Theory and Applications of Satisfiability Testing (SAT)*, pages 104–122. Springer, 2016.
- [9] J. Elffers and J. Nordström. Divide and conquer: Towards faster pseudo-Boolean solving. In *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence (IJCAI-ECAI)*, pages 1291–1299, 2018.
- [10] M. Anders, S. Brenner, and G. Rattan. satsuma: Structure-based symmetry breaking in SAT. In *Theory and Applications of Satisfiability Testing (SAT)*, volume 305 of *LIPICs*, 2024.
- [11] A. Biere, T. Faller, K. Fazekas, M. Fleury, N. Froleyks, and F. Pollitt. CaDiCaL 2.0. In *Computer Aided Verification (CAV)*, 2024.
- [12] W. Wang, Y. Hu, M. Tiwari, S. Khurshid, K. McMillan, and R. Miikkulainen. NeuroBack: Improving CDCL SAT solving using graph neural networks. In *International Conference on Learning Representations (ICLR)*, 2024.
- [13] J. E. Reeves. *Cardinality Constraints in Boolean Satisfiability Solving*. PhD thesis, Carnegie Mellon University, 2025.
- [14] C. Yu, R. Liang, C.-T. Ho, and H. Ren. Autonomous code evolution meets NP-completeness. arXiv:2509.07367, 2025.